

Bilgisayar Virüsleri ve Güvenlik Modelleri

CEN429 Güvenli Programlama — Hafta 2

Dr. Öğr. Üyesi Uğur CORUH · 25.09.2026

Bugünün planı (3 saat)

Saat	Konu
1	Zararlı yazılım: tarih, anatomi, türler · gizlenme
2	Demo 1–2 · karşı önlemler · saldırı ağacı (Demo 4) · erişim ve modeller (Demo 3, 6)
3	CWE · OWASP · CVE · CVSS (Demo 5) · yaşam döngüsü · proje

Öğrenme çıktısı: ÖÇ.1 (yaygın yazılım güvenlik açıklarını tanımlar ve sınıflandırır)

Önceki haftalardan gelenler

- **Varlık, tehdit, zafiyet, risk ve saldırgan modeli** — bir varlığın değerini, ona yönelik tehdidi, zafiyeti ve olasılık $\times$ etki olan riski ayırt etme dili (**Hafta 1**)
- **STRIDE ve saldırı ağacı** — bir arayüzdeki tehditleri STRIDE'in altı harfiyle sorup kök hedeften dallara VE/VEYA mantığıyla ayrıştırma yöntemi (**Hafta 1**)

Bu hafta: bu dili zararlı yazılım (tehdit) ve erişim modelleri (savunma) üzerinden somutlaştırıyoruz; saldırı ağacına **maliyet** ekleyip en ucuz saldırı yolunu hesaplıyoruz (Demo 04).

Bu haftanın kavramları

Her terim, gövdede ilk geçtiği yerde tanımlanır; burada yalnız **nerede** olduğunu işaretliyoruz.

Kavram	Nerede
Zararlı yazılım (malware)	Bölüm 1
Bir virüsün üç parçası	Bölüm 1
Zararlı yazılım türleri	Bölüm 1
Gizlenme: polimorfik/metamorfik	Bölüm 2
Tespit yöntemleri	Bölüm 2
Entropi	Bölüm 2
Denetim kaydı (audit log)	Bölüm 3
Erişim denetimi	Bölüm 4
DAC / MAC / RBAC	Bölüm 4
Bell–LaPadula, Biba, Clark–Wilson	Bölüm 4
CWE	Bölüm 5
CVE ve CVSS	Bölüm 5
OWASP Top 10 ve MASVS	Bölüm 5
Zafiyet yaşam döngüsü	Bölüm 5

Demolar nasıl çalışıyor?

- Tek kaynak, iki platform: **Windows (Visual Studio 2022 Community, MSVC)** ve **WSL/Linux (GCC)**
- Özet/şifreleme gerektiren demolarda Windows'ta işletim sisteminin **BCrypt** kütüphanesi, Linux'ta **OpenSSL** kullanılır (`code/common/cen429_crypto.h`) — Windows'ta ayrıca OpenSSL kurmanız gerekmez
- Derle: Windows `.\build.ps1` · WSL/Linux `./build.sh`
- Çalıştır: her demo klasöründe Windows `.\demo.ps1` · WSL/Linux `sh demo.sh`
- **Demo 6** (TOCTOU) yalnız WSL/Linux'ta çalışır

 **Etik:** Hiçbir demoda gerçek zararlı yazılım, kendini çoğaltan kod, dosya silme/şifreleme ya da ağ saldırısı yoktur. Hepsi kendi klasöründe, yönetici yetkisi olmadan çalışan **güvenli benzetimlerdir**; öğrendiklerinizi yalnız kendi bilgisayarınızda deneyin.

1. Zararlı yazılım: türler ve tarih

"Virüs" her şey değildir

Zararlı yazılım (malware): sahibinin izni dışında zarar veren her program.

- Virüs, zararlı yazılımın **yalnızca bir türüdür.**
- Ayrım **nasıl yayıldığına** göre yapılır:
 - **Virüs:** başka dosyaya iliştilir, taşıyıcı çalışınca yayılır
 - **Solucan:** kendi başına, ağ üzerinden yayılır
 - **Truva atı:** masum görünür, kendini kopyalamaz
 - **Fidye / rootkit / bot / silici:** yük ya da davranış türleri

Haftanın üç parçası — şema

Bu haftanın üç parçası

tehdidi tanı → savun → ortak dille adlandır



Üçü birlikte: ne olduğunu bilmeden savunamazsın, adlandırmadan yönetemezsin.

- **Tehdit:** zararlı yazılım türleri ve gizlenmesi
- **Savunma:** erişim denetimi ve biçimsel modeller
- **Ortak dil:** CWE · CVE · CVSS · OWASP · MASVS

Kısa tarihçe — zararlı yazılım ve güvenlik modelleri

- **1949** — von Neumann: **kendini çoğaltan** otomata (virüsün matematiksel kökü)
- **1971** Creeper · **1986** Brain (ilk PC virüsü) · **1988 Morris Worm** interneti durdurur
- **1973–77** — **Bell–LaPadula** (gizlilik), **Biba** (bütünlük); **1987** Clark–Wilson
- **1999 → 2006** — **CVE** · **CWE** · **CVSS**: ortak sınıflandırma dili

İki ayrı kol: **zararlıyı tanıma** + **erişimi modelleme**. Bugün ikisini birden görüyoruz.

Zararlı yazılımın kısa tarihi — şema

Zararlı yazılımın kısa tarihi

şaka ve merak → suç → devlet operasyonu

Creeper — ilk
kendini kopyalayan

1971

Morris solucanı —
internet durdu

1988

Stuxnet — devlet
düzeyinde

2010

1986

Brain — ilk PC
virüsü

2003

Slammer —
10 dk'da
küresel

2017

WannaCry —
fidye + solucan

Değişen şey teknik değil, SAİK: her dönemde saldırganın amacı farklı.

Kısa tarih: dönüm noktaları

Yıl	Olay	Türü
1984	Cohen'in tanımı	Kuram: "tam tespit imkânsız"
1986	Brain	İlk yaygın PC virüsü (önyükleme sektörü)
1988	Morris	İnternette yayılan ilk büyük solucan
1989	Brewer–Nash	Chinese Wall: çıkar çatışması modeli
1999	Schneier	Saldırı ağaçları (Attack Trees)
2000	ILOVEYOU	E-posta ekiyle milyonlar
2001	Code Red	Ağ açığıyla bellekte yayılan solucan
2003	SQL Slammer	10 dakikada yarısı — en hızlı yayılan solucan
2010	Stuxnet	Endüstriyel denetleyiciyi hedef alan solucan
2017	WannaCry	Kendi kendine yayılan fidye yazılımı
2017	NotPetya	Fidye gibi görünen silici (yıkıcı)
2019/2023	CVSS v3.1 / v4.0	Ciddiyet puanlama standardı, iki nesil

Cohen 1984: tespiti sınırı

Bir virüs, kendini (belki değiştirerek) başka programlara **kopyalayabilen** programdır.

- Cohen kanıtladı: "**Bu program virüs mü?**" sorusunu her durumda doğru yanıtlayan genel bir algoritma **yoktur** (durma probleminden indirgenir).
- Pratik sonuç: hiçbir antivirüs %100 tespit edemez.
 -  **Yanlış pozitif:** temiz dosyayı yakalar
 -  **Yanlış negatif:** zararlıyı kaçırır

Bunu Demo 01'de kendi gözümüzle göreceğiz.

Bir virüsün üç parçası

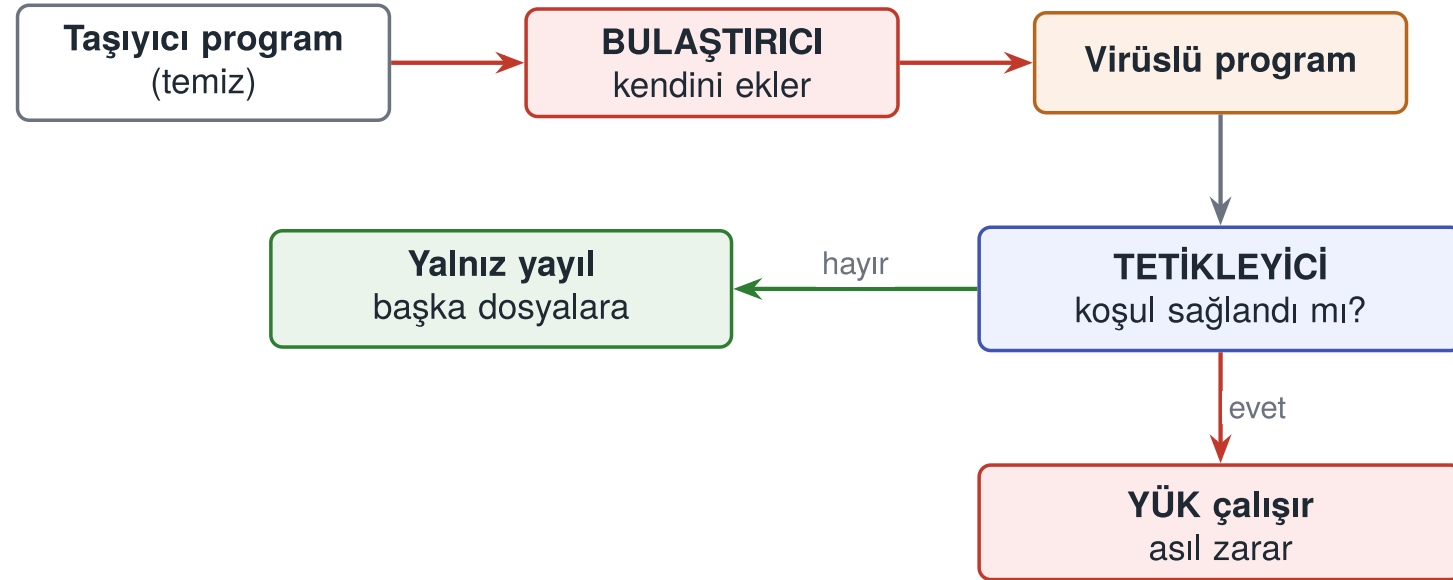
Parça	İşi	Hastalık benzetmesi
Bulaştırıcı	Kendini kopyalar	Hücreye girip çoğalma
Tetikleyici	"Ne zaman?" koşulu	Kuluçka süresi
Yük	Asıl zarar	Semptomlar

- Tetikleyici koşulu bekleyip patlayan koda **mantık bombası** denir.
- Her parça zorunlu değildir: yalnız yayılan, yüksüz bir virüs de vardır (yine de zafiyettir).

Virüsün üç parçası — şema

Bir virüsün üç parçası

bulaştırıcı · tetikleyici · yük



“Ne zaman?” sorusuna TETİKLEYİCİ bakar; koşula bağlı patlayana *mantık bombası* denir.

Virüs alt türleri

- **Program (dosya) virüsü:** çalıştırılabilir dosyaya bulaşır; dosya çalışınca virüs de çalışır.
- **Makro virüsü:** ofis belgesinin makro diline yazılır; belge açılınca çalışır (Melissa, 1999).
- **Önyükleme sektörü virüsü:** diskin ilk sektörüne bulaşır, işletim sisteminden **önce** çalışır (Brain, 1986).
 - Modern **UEFI Secure Boot** bu sınıfı büyük ölçüde zorlaştırdı.

Solucan, truva atı, fidye

Tür	Yayılma	Örnek
Solucan	Kendi başına, ağdan; kullanıcı gerekmez	Morris, Code Red, WannaCry
Truva atı	Masum görünür; kullanıcı çalıştırır; kopyalanmaz	"Bedava oyun", sahte güncelleme
Fidye yazılımı	Dosyaları şifreler, para ister	CryptoLocker, WannaCry

Solucanın tehlikesi hızdır: dakikalar içinde yüz binlerce makine.

Diğer türler

- **Rootkit:** kendini ve diğer zararlıları işletim sisteminden **gizler** (dosya/süreç/ağ listesinden siler). Tespiti en zor sınıf.
- **Bot / botnet:** uzaktan komut alan makineler ağı (DDoS, spam, madencilik).
- **Casus / reklam yazılımı:** veri toplar ya da reklam gösterir.
- **Silici (wiper):** fidye gibi görünür ama veriyi **kalıcı yok eder** (NotPetya). Amaç para değil, yıkım.
- **Arka kapı (backdoor):** gizli erişim bırakır.

Solucan ne kadar hızlı yayılır?

SI modeli — duyarlı (S) ve enfekte (I) makineler:

$$i(t+dt) = i(t) + \text{beta} * i(t) * (1 - i(t)/N) * dt$$

- $\text{beta} * i$: enfekte çoğaldıkça tarama hızlanır → **üstel** büyüme
- $(1 - i/N)$: duyarlı azaldıkça isabet düşer → **doygunluk**
- Sonuç: sessiz açılış → ani patlama → doygunluk (S eğrisi)

⚠ Savunmacının tek penceresi **açılış evresi**

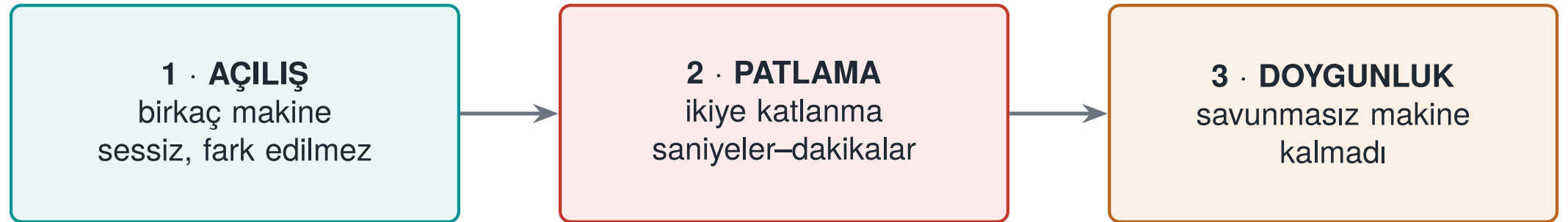
SI modeli: adım adım

- **S (duyarlı):** henüz bulaşmamış, savunmasız makine.
- **I (enfekte):** zaten bulaşmış, kendisi de tarayan makine.
- **N:** toplam savunmasız makine sayısı ($S + I$).
- Her enfekte makine birim zamanda **rastgele adres tarar**; adres duyarlıya denk gelirse onu da enfekte eder.

Salgın modeli — şema

Solucan salgını: SI modeli

yayılma üstel — insan müdahalesi yetişemez



Morris (1988) ve Slammer (2003): dakikalar içinde internetin büyük kısmı etkilendi.

Demo 09 — Salgın simülasyonu

code/week-02/09-outbreak-simulation · yalnız hesaplama, ağ yok

Senaryo: Slammer benzeri (rastgele tarama, hızlı UDP)

54 s	78	0.1%	
1.5 dk	1430	1.9%	#.....
2.4 dk	45601	60.8%	#####.....
3.3 dk	74429	99.2%	#####

%90 doygunluk: Code Red ~13.3 sa · Slammer ~2.7 dk · Hitlist ~50 s

- Code Red (2001): ~359.000 sunucu · ikiye katlanma ~37 dk
- Slammer (2003): tek 404 baytlık UDP paketi · ~8,5 s'de ikiye katlanma
- ☒ Ders: insan hızında tepki yetişmez → **önceden yama, kapalı port, otomatik savunma**

Gerçek sayılar: Code Red vs Slammer

Olay	Giriş yolu	Hız
Code Red (2001)	IIS <code>.ida</code> taşması (CVE-2001-0500)	~359.000 sunucu; ikiye katlanma ~37 dk
SQL Slammer (2003)	SQL Server ayrıştırma hizmeti, tek 404 baytlık UDP paketi (CVE-2002-0649)	~8,5 sn'de ikiye katlandı; ~10 dk'da %90

Slammer'ı hızlandıran iki şey: **tek paket** yeterliydi (bağlantı kurma yok), hedefi **olabildiğince hızlı** taradı.

Slammer neden yavaşladı?

- İlk dakikadan sonra yayılma hızı düştü.
- Neden? **Savunma değil** — ağın **bant genişliği doldu**.
- Solucan kendi trafiğiyle kendi yayılma kanalını tıkadı.
- Ders: bazen saldırganın kendi başarısı, kendi sınırını yaratır.

İkiye katlanma süresi

- Enfekte sayısının **iki katına çıkması** için geçen süre, bir salgının hızının en sezgisel ölçüsüdür.
- Code Red: ikiye katlanma ~**37 dakika**.
- Slammer: ikiye katlanma ~**8,5 saniye**.
- Aradaki fark ~260 kat — ikisi de "solucan" olsa da savunma penceresi **tamamen farklıdır**.

Demo 09 çıktısını okurken: açılış evresi

- İlk 54 saniyede yalnız **78 makine** enfekte (%0.1).
- Grafikte neredeyse hiçbir şey görünmüyor.
- 1,5 dakika sonra patlama **başlıyor**.
- "Şimdilik az" demek, üstel büyümede **en pahalı yanılgıdır**.

Demo 09 çıktısını okurken: hız ve hitlist

- **Hız (beta):** tarama hızını artırmak, doygunluğu **saatlerden dakikalara** indirir.
- Bu yüzden savunma **otomatik** olmalı: güvenlik duvarı kuralı, ağ bölümleme, önceden yama.
- **Hitlist:** saldırgan hazır hedef listesiyle başlarsa açılış evresi **atlanır**.
- Savunmacının zaten dar penceresi tamamen **kapanır**.

SI salgın modeli: yayılma adım adım (animasyon)

N=16 savunmasız makine, i0=1 başlangıç, beta=0.50



#####... 14/16 (88%)

Büyüme en hızlı %50 dolayında oldu — sonrasında yavaşladı, çünkü kalan hedef sayısı azaldı.

Salgın — uç durum: başlangıçta zaten yarısı bulaşmış

N=16 savunmasız makine, i0=8 başlangıç, beta=0.40



16/16 (100%)

Büyüme en hızlı %50 dolayında oldu — sonrasında yavaşladı, çünkü kalan hedef sayısı azaldı.

Programcıya ders (salgın)

- Yayılma hızını programcı belirlemez.
- Ama **ilk bulaşmayı mümkün kılan hatayı** programcı yapar.
- Code Red, Slammer, Blaster — hepsi **sınır denetimsiz** bir taşma ile yayıldı.
- Her dinlenen ağ portu bir **saldırı yüzeyidir**; kullanılmayan hizmeti hiç açmayın.

Kendiniz deneyin: beta yarıya inerse?

`outbreak.c` içindeki `beta` değerini yarıya indirip yeniden çalıştırın.

Soru: Doygunluk süresi nasıl değişir? Hitlist senaryosunda `i0` 'ı 100'e çıkarınca fark ne kadar kalır?

Cevap: Beta yarıya inince büyüme yavaşlar, doygunluk süresi **uzar**. Hitlist senaryosunda açılış evresi zaten atlandığı için `i0` artışının etkisi **daha küçüktür** — hitlist'in asıl gücü açılış atlaması, başlangıç sayısı değil.

2. Yayılma ve gizlenme

Gizlenme merdiveni

Yöntem	Ne değişir?	Savunma
Şifreleme	Gövde şifreli, çözücü sabit	Çözücüü imzala
Oligomorfizm	Çözücünün birkaç biçimi	Hepsini imzala
Polimorfizm	Çözücü her kopyada yeniden üretilir	Emülasyonla çöz
Metamorfizm	Şifreleme yok, kodun tamamı yeniden yazılır	Davranışa bak

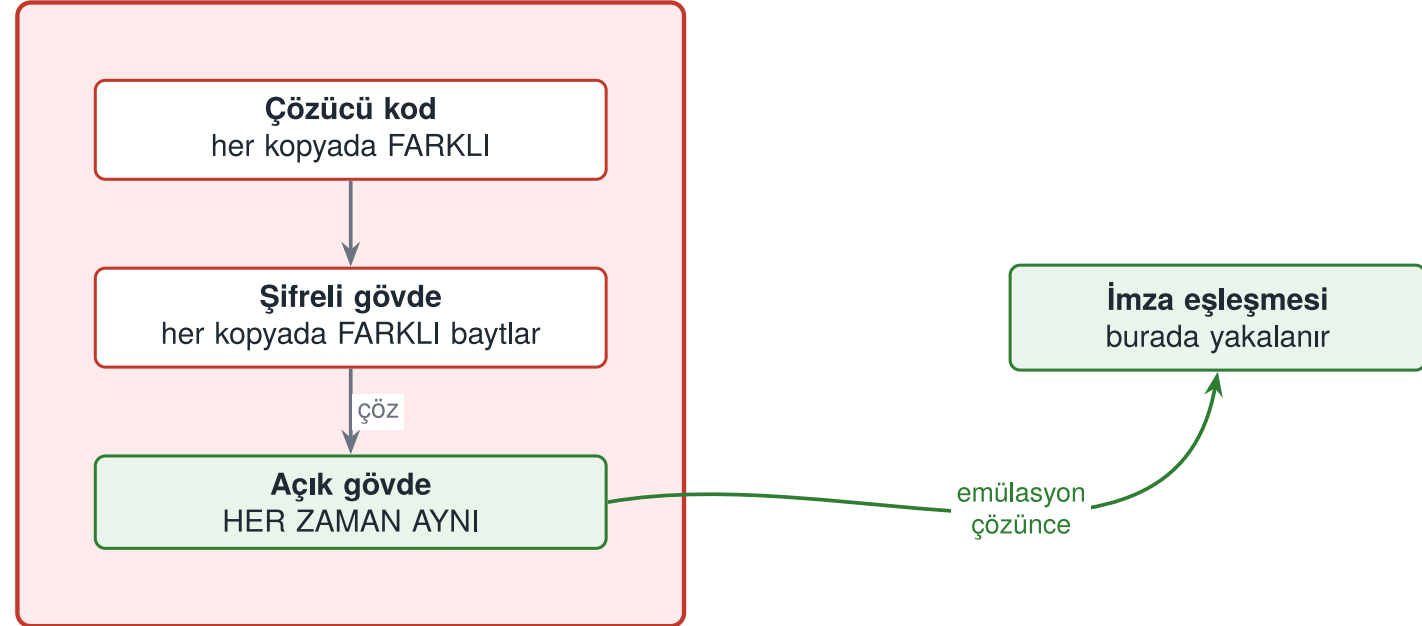
Anahtar fikir: şifreli gövde her kopyada farklı görünür, **çözülünce hep aynıdır.**

Şifreli / polimorfik virüsün yapısı

Polimorfik virüs neden imzayla yakalanmaz?

her kopyada baytlar değişir, açılan gövde aynı kalır

Şifreli / polimorfik virüs



Bu yüzden emülasyon kullanılır: virüsü yalıtılmış ortamda çalıştırıp AÇIK gövdeyi imzalar.

Emülasyon tam da bu son kutuyu görmek için çözücüyü çalıştırır.

Demo 01 — Oyuncak antivirüs

code/week-02/01-signature-scanner · Kitap: Tarif 12.1

- Zararsız, çalıştırılmayan bir "**KAPSÜL**" biçimi: başlık + çözücü + gövde.
- Dört yöntem, dört sınır:

Yöntem	Yakalar	Atlatılır
Hash	Birebir aynı dosya	Tek bayt değişince
Desen	Küçük değişiklik	Çözücü değişince (polimorfizm)
Sezgisel	İmzasız	Yanlış pozitif/negatif
Emülasyon	Hepsi	Anti-emülasyon

Demo 01 — Hash kırılma, desen dayanıklı

STEP 2 (hash):

```
sample_a.bin      CAUGHT  Sample.BlueCat.A
sample_a_1byte.bin CLEAN   <- one byte changed, digest is different
```

STEP 3 (pattern):

```
poly_1.bin  CAUGHT  Sample.Decoder.xs32
poly_2.bin  CAUGHT  Sample.Decoder.xs32
poly_3.bin  CLEAN   <- the decoder changed too (polymorphism)
```

Hash çığ etkisiyle tek baytta kırılır; **desen** çözücüyü yakalar ama polimorfizm onu da atlatır.

Demo 01 — Sezgisel: iki tür hata

STEP 5 (heuristic):

sample_a.bin	CLEAN	score= 0	<- FALSE NEGATIVE (missed it)
poly_1.bin	SUSPICIOUS	score= 80	
archive.bin	SUSPICIOUS	score= 50	<- FALSE POSITIVE (harmless)

- İmza olmadan polimorfiği yakaladı 
- Ama zararsız yüksek-entropili dosyayı şüpheli sandı 
- Ve düz gövdeli örneği kaçırdı 

Cohen'in belirsizliği tam burada.

Demo 01 — Emülasyon hepsini çözer

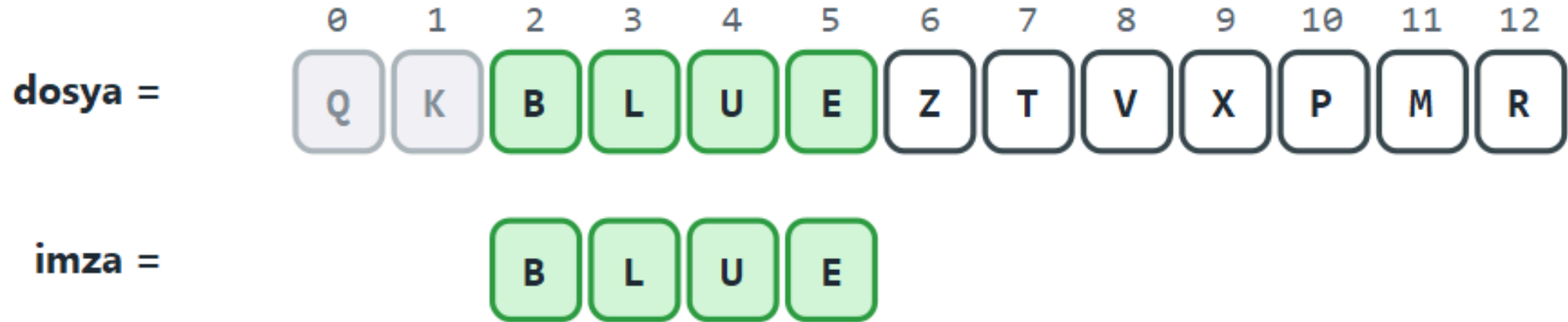
STEP 6 (emulation):

```
poly_1.bin  CAUGHT  ... | 5 commands, 0 NOP, xs32  
poly_2.bin  CAUGHT  ... | 5 commands, 0 NOP, xs32  
poly_3.bin  CAUGHT  ... | 8 commands, 4 NOP, lcg8
```

- Çözücüğü **güvenli sanal makinede** çalıştırır, açılan gövdeyi tarar.
- Üç polimorfik kopya da **aynı imzaya** döndü.
- Güçlü ama **pahalı**; zararlı "sanal makinedeyim" derse gizlenebilir.

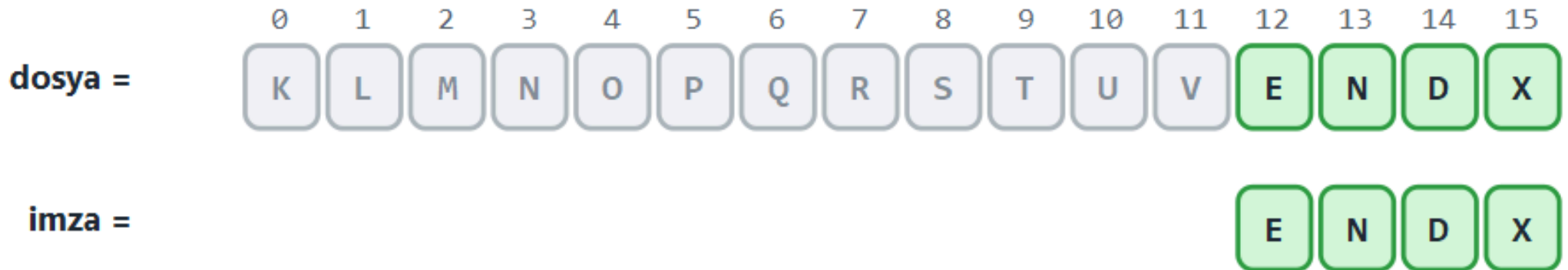
Sonuç: tek yöntem yetmez → katmanlı savunma.

İmza taraması: hash + desen + emülasyon (animasyon)



konum 2: BLUE == BLUE — İMZA BULUNDU.

İmza taraması — uç durum: imza dosyanın son konumunda



konum 12: ENDX == ENDX — İMZA BULUNDU.

Entropi: rastgelelik ölçüsü

$$H = - \sum_{b=0}^{255} p(b) \log_2 p(b) \quad (0 \dots 8 \text{ bit/bayt})$$

- **H ~ 0:** hep aynı bayt · **H ~ 4–5:** metin/kod
- **H ~ 6:** derlenmiş program · **H ~ 8:** şifreli/sıkışık/rastgele

Antivirüs: "paketli/şifreli kod?" · EDR: "dosyaları hızla şifreleyen süreç?" (fidye)

Demo 02 — Entropi ölçer

code/week-02/02-entropy

tekduze.txt	0.00		tekduze
belge.txt	4.17	#####.....	dusuk
kod.bin	6.23	#####.....	orta
sifreli.bin	7.95	#####	YUKSEK
rastgele.bin	7.95	#####	YUKSEK

⚠ Entropi tek başına "zararlı mı?" demez: `.zip` / `.png` de yüksektir. **Beklenmedik yerde** yüksekse ipucudur.

Demo 02 — Sıcak bölge ve fidye izi

Mixed file (window 512):

1024-1535 4.04 low

1536-2047 7.54 HIGH <- hidden encrypted section

2048-2559 7.57 HIGH

Ransomware trace: document.txt 4.06 --> encrypted.bin 7.94

- Paketlenmiş programın deseni: **küçük açıcı + yüksek entropili gövde**.
- Bir süreç kısa sürede çok dosyada bu sıçramayı yapıyorsa **davranış tabanlı** koruma alarm verir.

Entropi: pencereleli tarama (animasyon)

24 baytlık dosya, 8 baytlık pencerelerle taranıyor

dosya =

54 49 48 4F 58 52 4E 4A 56 44 56 46 4C 57 50 44

H=2.60 (low)

53 4A 20 42 59 42 44 46

H=2.75 (low)

Pencereler arasında büyük bir sıçrama yok: dosyanın karakteri baştan sona tutarlı.

Entropi — uç durum: pencere dosyayı bölmüyor

19 baytlık dosya, 8 baytlık pencerelerle taranıyor

dosya =



H=3.00 (low)



H=1.58 (low)

Pencereler arasında büyük bir sıçrama yok: dosyanın karakteri baştan sona tutarlı.

Karşı önlemler

Yöntem	Güçlü	Zayıf
İmza tabanlı	Hızlı, düşük yanlış pozitif	Bilinmeyen/polimorfiği kaçırır
Sezgisel	İmzasız yeni örnek	Yanlış pozitif/negatif
Davranış tabanlı	Gizlenmeye dirençli	Zarar başlamış olabilir
Kum havuzu	Gerçek davranışı görür	Yavaş; anti-VM
Emülasyon	Çözücüyü açar	Anti-emülasyon
İtibar / bulut	Hızlı yayılan tehdide iyi	Gizlilik, bağlantı

Modern EDR hepsini birleştirir, davranışa ağırlık verir.

Karşı önlem katmanları — şema

Karşı önlemler: dört katman, dört soru

her katman ötekinin kaçırdığını yakalar

İmza tabanlı

“bu dosyayı daha önce gördüm mü?” — bilinen zararlıyı yakalar

Sezgisel (heuristic)

“şüpheli görünüyor mu?” — benzerini yakalar

Davranış tabanlı

“ne YAPIYOR?” — yeni zararlıyı çalışırken yakalar

Bütünlük izleme

“değişti mi?” — beyaz liste, kurcalamayı yakalar

Tek katman yetmez: imza yeni, davranış sessiz kaçırır.

EDR örneğinde davranış izi

- Süreç, kısa sürede **yüzlerce dosyayı** okuyup yüksek entropili haliyle geri yazıyor.
- İçeriğe hiç **bakmadan**, yalnız bu davranış neredeyse kesin bir **fidye yazılımı** belirtisidir.
- Davranış tabanlı koruma dosyanın "ne olduğuna" değil, "ne yaptığına" bakar.
- Bu ipucu Demo 02'deki entropi sıçramasıyla **aynı fikrin** çalışma zamanı hâlidir.

Aynı alet kutusu: saldırgan ve savunmacı

- Bir zararlının **gizlenmek** için kullandığı teknik ile meşru yazılımın **kendini korumak** için kullandığı teknik çoğu zaman **aynıdır**.
- Fark **niyet ve bağlamdadır**: biri sizin varlığınızı korur, diğeri imzadan kaçar.
- Şifreli gövde + çözücü deseni: polimorfik virüste de, korumalı mobil uygulamada da var.
- Bu simetri dönem boyunca (9., 11. hafta) tekrar karşımıza çıkacak.

Örnek: mobil ödeme neden şifreli sabit tutar?

- Tersine mühendisliği **yavaşlatmak** için hassas dizeler/sabitler şifreli tutulur, kullanım anında çözülür.
- Bir değerlendirici binary dosyada **yüksek entropili bölge** görünce sorar:
 - Çözücü nerede?
 - Anahtarı nasıl buluyor?
 - Anahtar bellekte ne kadar açık kalıyor?
- Aynı entropi ölçümü hem antivirüsün hem değerlendiricinin **ortak aracıdır**.

İtibar / bulut tabanlı tespit

- Dosyanın **yaygınlığı, kaynağı, dijital imzası** bulutta sorgulanır.
- "Milyonlarca kullanıcıda görülmüş ve temiz" ise güvenilir sayılır; "hiç görülmemiş, imzasız" ise şüphelidir.
- Güçlü yanı: **hızlı yayılan** tehdide karşı anında güncellenir.
- Zayıf yanı: **gizlilik** (dosya bilgisi buluta gider) ve internet **bağlantısı** gerektirir.

Demo 07 — Kural tabanlı tespit

code/week-02/07-rule-engine · YARA benzeri mini motor, **kendi zararsız desenlerimiz**

```
rule Word_Context {  
  string $a      = "download"  
  string $b      = "run"  
  string $magic = "CAPSULE/1"  
  condition ($a and $b) and $magic  
}
```

Adım	Sonuç	Ders
Zararsız kılavuz	Word_Pair eşleşti	✗ Bağımsız kural → yanlış pozitif
Değişmiş varyant	yalnız joker (??) desen yakaladı	✓ Değişmeyen iskeleti hedefle
Bozuk kural dosyası	REDDEDİLDİ	✓ Tespit aracı da bir ayrıştırıcıdır

Kural motoru: koşul ayrıştırıcı (animasyon)

koşul = "(\$a and \$b) and \$magic"

sayımlar = \$a=1 \$b=1 \$magic=1

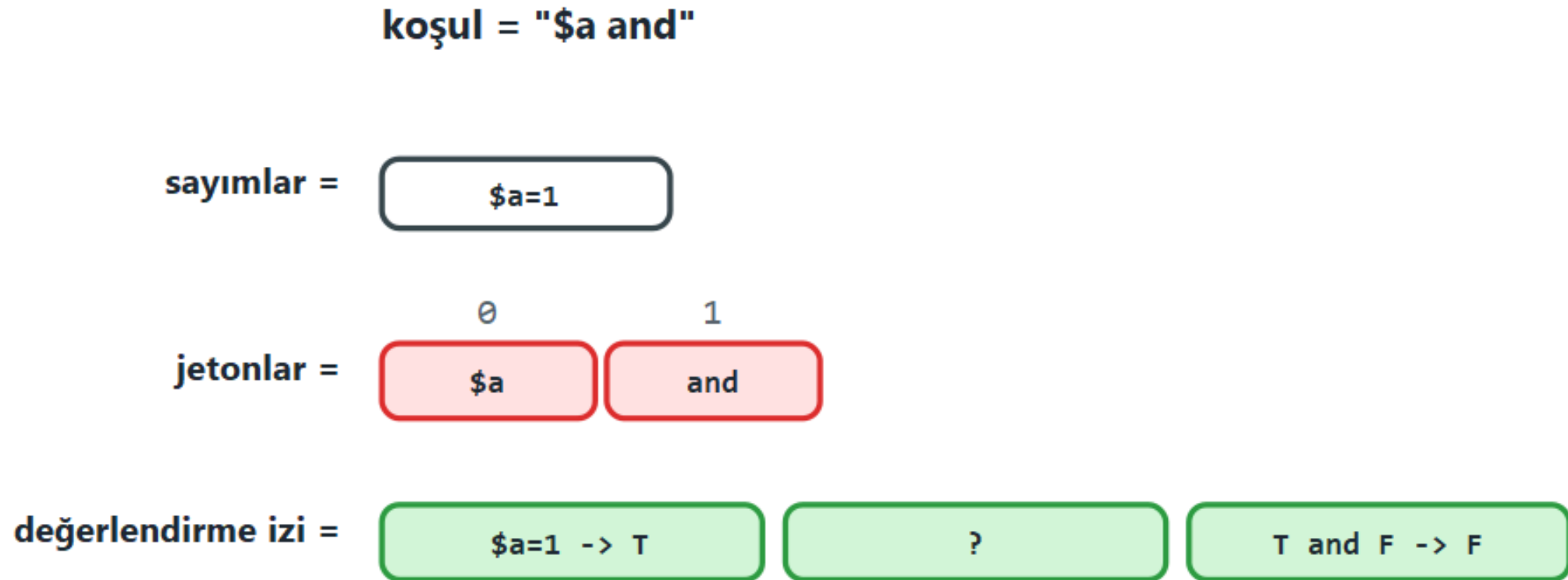
jetonlar =

0	1	2	3	4	5	6
(	\$a	and	\$b	)	and	\$magic

değerlendirme izi = \$a=1 -> T \$b=1 -> T T and T -> T (T) \$magic=1 -> T T and T -> T

>>> SONUÇ: koşul EŞLEŞTİ.

Koşul ayrıştırıcı — uç durum: kırık bir koşul



>>> SONUÇ: KOŞUL HATASI (sözdizimi bozuk ya da jeton arttı).

Kara liste vs beyaz liste

- **Kara liste:** "kötü olanı tanı" — bilinen zararlıyı listele, geri kalanına izin ver.
- **Beyaz liste:** "iyi olanı tanı" — bilinen iyiyi listele, **geri kalan her şeyden şüphelen.**
- Bütünlük izleme, beyaz listenin **dosya düzeyindeki** biçimidir.
- Sistem iyiysen her dosyanın özeti bir **manifeste** yazılır, sonra düzenli karşılaştırılır.

Soru — hangi tür, hangi katman?

E-posta ekindeki "**fatura.pdf.exe**" açılınca çalışıp adres defterindeki herkese kendini yolluyor.

Soru: Bu hangi tür zararlı? İlk yakalayacak katman ne olurdu?

Cevap: Solucan/truva karışımı — kullanıcı **açar** (truva atı gibi başlar), sonra **kendi kendine yayılır** (solucan gibi). İlk yakalayacak katman: **imza/desen** (bilinen bir örneğe) ya da **sezgisel** (yeniyse).

Soru — tedarik zinciri saldırısı

Bir yazılımın **resmî güncellemesine**, derleme sırasında arka kapı ekleniyor (SUNBURST benzeri).

Soru: İmza tabanlı tespit ve izin listesi (whitelisting) bunu yakalar mı?

Cevap: Hayır, ikisi de başarısız olur. Dosya **geçerli imzalıdır** ve izin listesinde zaten vardır (güncelleme resmî kaynaktan geliyor). Tedarik zinciri saldırısının korkutucu yanı tam budur: güvendiğiniz kaynağın kendisi ele geçirilmiştir.

Demo 08 — Bütünlük izleme (beyaz liste)

code/week-02/08-integrity-monitoring · SHA-256 manifesti + **HMAC mühür**

```
CHANGED library.bin 5ab02b71... -> 12c5803f... (48->62 B)
DELETED data.txt    (in the manifest, not in the folder)
NEW      hidden.bin  (not in the baseline)
-- attacker rewrites the manifest --
RESULT: integrity preserved    <- monitor FOOLED
SEAL INVALID -> DO NOT TRUST the manifest
```

✅ Özet değişikliği yakalar · ⚠️ **beklenen değerler de korunmalı** (HMAC/imza, anahtar ayrı yerde)

Tarif 12.2: CRC32 → SHA-256/HMAC · 6. haftada çalışma zamanı bütünlük denetimi

Değerlendirici: bütünlük denetimini nasıl kırar?

İki adımlı test:

1. Önce izlenen bir **dosyayı** değiştirir → tespit edildi mi?
2. Sonra **beklenen değerleri** (manifest, gömülü özet) de değiştirmeye çalışır.

İkinci test geçilemiyorsa denetim yalnız **kazaya** karşı korur, **saldırgana** karşı değil.

Sabit zamanlı karşılaştırma

- İki özeti karşılaştırırken **ilk farklı bayta** kadar mı, yoksa **hep aynı sürede** mi bakılıyor?
- Süre farkı, doğru baytların **sayısını sızdırabilir** (zamanlama yan kanalı).
- Bütünlük ve HMAC karşılaştırmaları **sabit zamanlı** yapılmalı.
- Demo 08'in HMAC mührü bu yüzden özel bir karşılaştırma kullanır.

Neden CRC32 yeterli değil?

- **CRC32** hata tespiti için tasarlandı, **saldırgana karşı değil**.
- Aynı CRC32 değerini veren **farklı içerik** bulmak kolaydır (çakışma ucuz).
- Saldırgan dosyayı değiştirip CRC'yi **eski değere döndürebilir**.
- Bu yüzden bütünlük denetiminde **kriptografik özet** (SHA-256) ya da **imza** gerekir.

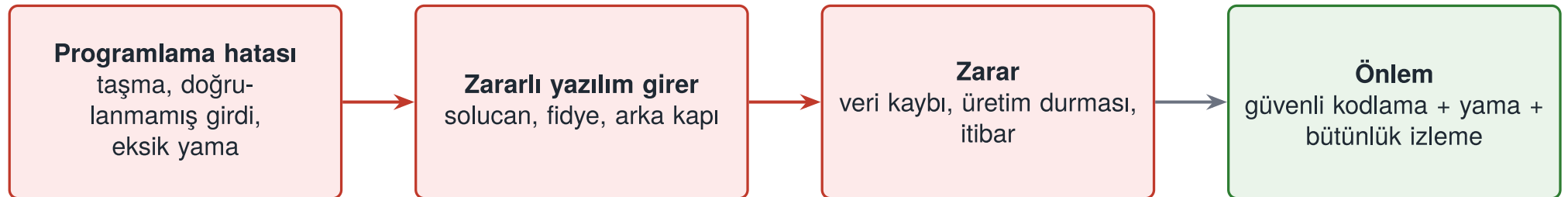
Olaylar: içeri nasıl girdiler?

Olay	Giriş yolu	Hata sınıfı
Morris (1988)	<code>fingerd</code> <code>gets()</code> taşması · <code>sendmail</code> DEBUG · <code>rsh</code> güveni	CWE-120/242 · 489 · 287
Code Red (2001)	IIS <code>.ida</code> taşması (CVE-2001-0500)	CWE-120
Slammer (2003)	SQL çözümleme hizmeti, tek UDP paketi (CVE-2002-0649)	CWE-121
Blaster (2003)	RPC/DCOM taşması (CVE-2003-0352)	CWE-120
Conficker (2008)	Server hizmeti yol işleme (CVE-2008-4250) · zayıf parola	CWE-119 · 521
WannaCry (2017)	SMBv1 (CVE-2017-0144) · yama 2 ay önce çıkmıştı	bellek güvenliği
NotPetya (2017)	güncelleme sunucusu üzerinden dağıtım	CWE-494
SUNBURST (2020)	derleme ortamına arka kapı	tedarik zinciri

Olay incelemelerinin iskeleti — şema

Olay incelemelerinin ortak iskeleti

Morris'ten WannaCry'a kadar aynı zincir



Her olay incelemesinde bu dört kutuyu doldurun; ders zincirin neresinde kırıldığıdır.

Olaylardan programcıya dersler

1. **Sınır denetimi:** 1988–2008 salgınlarının çoğu tek bir arabellek taşması (1. ve 4. hafta)
2. **Sürümde hata ayıklama kodu bırakma** (Morris / `sendmail` DEBUG)
3. **Gereksiz hizmeti açma:** her port bir saldırı yüzeyi (Slammer, Blaster)
4. **İmzalı güncelleme + istemcide doğrulama** (NotPetya) — 3. ve 10. hafta
5. **Derleme ortamı ve tedarik zinciri güvenliği** (SUNBURST) — 5. hafta SBOM
6. **Güncelleme mekanizması güvenlik özelliğidir:** yama gecikmesi = açık kapı

Stuxnet (2010): devlet düzeyinde saldırı

- Hedef: endüstriyel denetleyiciler (**PLC**) — bir uranyum zenginleştirme tesisi.
- Giriş: bir **kısayol (.lnk)** dosyasının görüntülenmesiyle kod yükleme + **dört** sıfırıncı gün açığı.
- **Çalınmış sürücü imzaları** kullanıldı — imzalı olduğu için güvenilir görüldü.
- Programcıya ders: görüntüleme sırasında kod çalıştırmayın; imza anahtarlarını donanımda (HSM) koruyun.

Eğilim: 1988–2008 vs 2017 sonrası

- **1988–2008:** salgınların çoğu **tek bir arabellek taşmasıyla** yayıldı (Morris → Conficker).
- **2017 sonrası:** giriş kapısı **güncelleme ve derleme süreçlerine** kaydı (NotPetya, SUNBURST).
- Saldırgan artık kodu kırmak yerine, kodun **size ulaştığı yolu** ele geçiriyor.
- İki savunma ucu: sınır denetimi (1., 4. hafta) + imzalı güncelleme/tedarik zinciri (3., 5., 10. hafta).

WannaCry: üç ders bir olayda

Konu	WannaCry'da ne oldu?	Bu haftaki bağlantı
Yayılma	SMB açığı (EternalBlue) ile kullanıcı hiçbir şey yapmadan solucan gibi yayıldı	Solucan tanımı, salgın modeli
Yük	Dosyaları şifreleyip fidye istedi	Entropi sıçraması (Demo 02)
Yama boşluğu	Yama saldırıdan haftalar önce çıkmıştı	Güncelleme = güvenlik özelliği

Tek bir olay, bu haftanın üç ana konusunu birleştiriyor.

Yama gecikmesi bir güvenlik açığıdır

- WannaCry'ın açığının yaması, saldırıdan **iki ay önce** yayımlanmıştı.
- Slammer'inki de **aylar önceden** vardı.
- Sonuç: güncelleme mekanizması **güvenlik özelliğinin bir parçasıdır**.
- Güncellemesi zor/yavaş bir ürün, düzeltilmiş hataları bile **aylarca açık tutar**.

Tartışma: makrolar neden varsayılan kapalı?

Soru: Office, 2022'den beri internetten gelen belgelerdeki makroları varsayılan engelliyor. Bu, Melissa ve ILOVEYOU gibi eski salgınların hangi zayıflığını kapatıyor?

Cevap: Kullanıcının "makroları etkinleştir" tuzağına düşmesine dayanan yayılma yolunu kapatıyor. **Varsayılan ayar**, kullanıcı eğitiminden daha güçlüdür: eğitim her kullanıcıda işe yaramayabilir, ama güvenli varsayılan **herkesi** aynı anda korur.

Tedarik zinciri: SUNBURST'ten NotPetya'ya

- **NotPetya (2017):** bir muhasebe yazılımının güncelleme sunucusu ele geçirildi, on binlerce makineye yayıldı.
- **SUNBURST (2020):** bir izleme yazılımının **derleme ortamına** sızılıp imzalı kütüphaneye arka kapı eklendi.
- Ortak nokta: saldırgan **sizin güvendiğiniz kaynağı** hedef alıyor, sizi değil.
- Savunma: güncellemeleri imzala + istemcide doğrula, derleme ortamını koru (5. hafta SBOM).

Olaylardan çıkan tek satırlık dersler

Olay	Programcı ne yapmalıydı?
Morris	Sınır denetimli okuma, hata ayıklama kodunu sürümde bırakmama
Code Red / Slammer / Blaster	Ağ girdisinin uzunluğunu doğrulama, gereksiz hizmeti kapatma
WannaCry	Ayrıştırıcıda sınır denetimi, eski protokolü kapatma, yamayı geciktirmeme
NotPetya	Güncellemeleri imzalama ve istemcide doğrulama
SUNBURST	Derleme ortamını koruma, tekrarlanabilir derleme

Yanılgı: "Her zararlı bir virüstür"

Yanılgı: Bulaşan her şeye "virüs" denir.

Gerçek: Virüs yalnız **bir türdür** — taşıyıcıya iliştilir, taşıyıcı çalışınca çalışır. Solucan taşıyıcı gerektirmez; truva atı kendini kopyalamaz.

Türü yanlış koymak, yanlış savunmayı seçtirir: solucana karşı **ağ yaması** gerekirken truva atına karşı **kullanıcı eğitimi** gerekir.

Yanılğı: "İmza veritabanım güncel, güvendeyim"

Yanılğı: Güncel imza veritabanı yeterli korumadır.

Gerçek: İmza yalnız **bilineni** yakalar. Tek baytlık değişiklik özet imzasını, çözücüsü değişen polimorfik kopya **desen** imzasını atlatır (Demo 01).

İmza, savunmanın **başlangıcıdır**, bitmez.

Yanılgı: "Yüksek entropi = zararlı"

Yanılgı: Yüksek entropili her dosya şüphelidir.

Gerçek: Şifreli, sıkıştırılmış ve rastgele veri hep **8'e yakındır**; `.zip`, `.png`, şifreli yedek de öyle.

Entropi yalnız **beklenmedik yerde** anlamlı bir ipucudur (Demo 02).

Yanılgı: kum havuzu ve emülasyonun sınırı

"Kum havuzunda temiz çıktı, o hâlde temiz." Zararlı sanal ortamı **sezip uyuyabilir**; sonuç "bu koşullarda bir şey yapmadı" demektir, "zararsızdır" değil.

"Emülasyon metamorfik zararlıyı da çözer." Emülasyon şifreli gövdenin çözüldüğü anı yakalar; metamorfik kodda **sabit bir gövde yoktur**. Burada davranış tabanlı tespit gerekir.

Yanılgi: "%100 tespit eden ürün alırlz"

Yanılgi: Doğru ürünü seçersek hiçbir zararlı kaçmaz.

Gerçek: Cohen'in (1984) kanıtladığı gibi, "bu program virüs mü?" sorusunu **her durumda doğru** yanıtlayan genel bir algoritma **yoktur**.

Her tespit yöntemi yanlış pozitif ile yanlış negatif arasında bir **denge** kurar.

Yanılğı: "Bütünlük izlemesini düz SHA-256 listesiyle yaptım"

Yanılğı: Bir dosya listesi + özetleri yeterli bütünlük denetimidir.

Gerçek: Saldırgan dosyayı değiştirdiği gibi **listeyi de** güncelleyebilir.

Beklenen değerler **anahtarlı** (HMAC) ya da **imzalı** olmalı ve saldırırganın erişemeyeceği yerde durmalıdır (Demo 08).

Yanılgı: "Yama yayımlandı, iş bitti"

Yanılgı: Yama çıktığı an risk biter.

Gerçek: WannaCry'ın kullandığı açığın yaması saldırıdan **yaklaşık iki ay önce** çıkmıştı.

Yama **uygulanana kadar** risk sürer; yayımlanan yama saldırgana da **yol gösterir**.

Kontrol listesi — zararlı yazılım ve tespit

- [] Virüs, solucan, truva atı, fidye, rootkit, bot, casus yazılım ve siliciyi **yayılma** ve **amaç** ile ayırabiliyorum.
- [] Bir virüsün bulaştırıcı, tetikleyici, yük parçalarını bir senaryoda gösterebiliyorum.
- [] Şifreli → oligomorfik → polimorfik → metamorfik merdiveninin her basamağını açıklayabiliyorum.
- [] Özet, desen, sezgisel, davranış/EDR, kum havuzu ve emülasyonun **güçlü/zayıf** yanını söyleyebiliyorum.
- [] SI salgın modelinde açılış/patlama/doygunluk evrelerini ve hitlist etkisini gösterebiliyorum.
- [] Morris, Code Red, Slammer, Stuxnet, WannaCry, NotPetya ve SUNBURST'ün **birer dersini** söyleyebiliyorum.

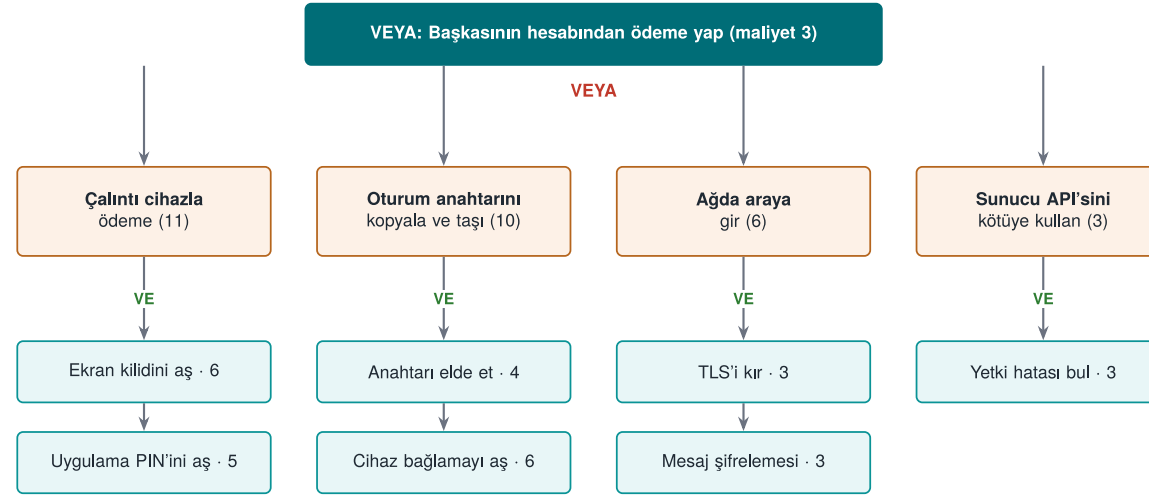
3. Saldırı ağaçları ve denetim kaydı

Saldırı ağacı = niceliksel tehdit

- Kök: saldırganın **hedefi** · Dallar: ona ulaşma yolları.
- **VEYA**: en ucuz çocuk yeter · **VE**: çocukların toplamı gerekir.
- Yapraklara **maliyet** yazarsak en ucuz saldırıyı buluruz.

Saldırı ağacı: başkasının hesabından ödeme

parantez içindeki sayı = saldırganın maliyeti (düşük = kolay)

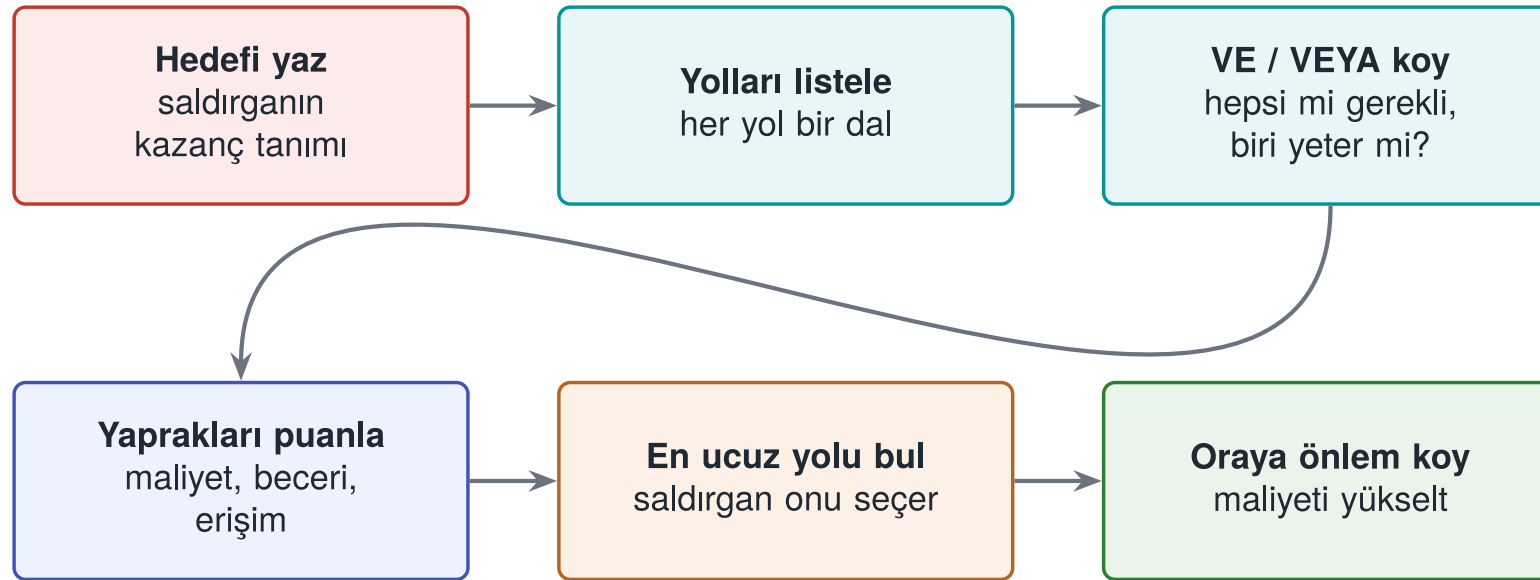


En ucuz dal (3) önce kapatılır: savunma bütçesi en olası yola harcanır.

Saldırı ağacı kurma adımları — şema

Saldırı ağacı nasıl kurulur?

niteliksel korkudan niceliksel karara



Ağaç, “neyi önce düzelteyim?” sorusuna sayıyla cevap verir.

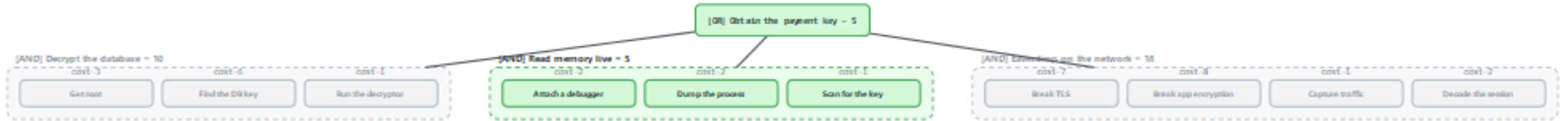
Demo 04 — Savunma maliyeti artırır

code/week-02/04-attack-tree

BEFORE: CHEAPEST ATTACK = 2 (read from memory)
AFTER: CHEAPEST ATTACK = 8 (RASP added)

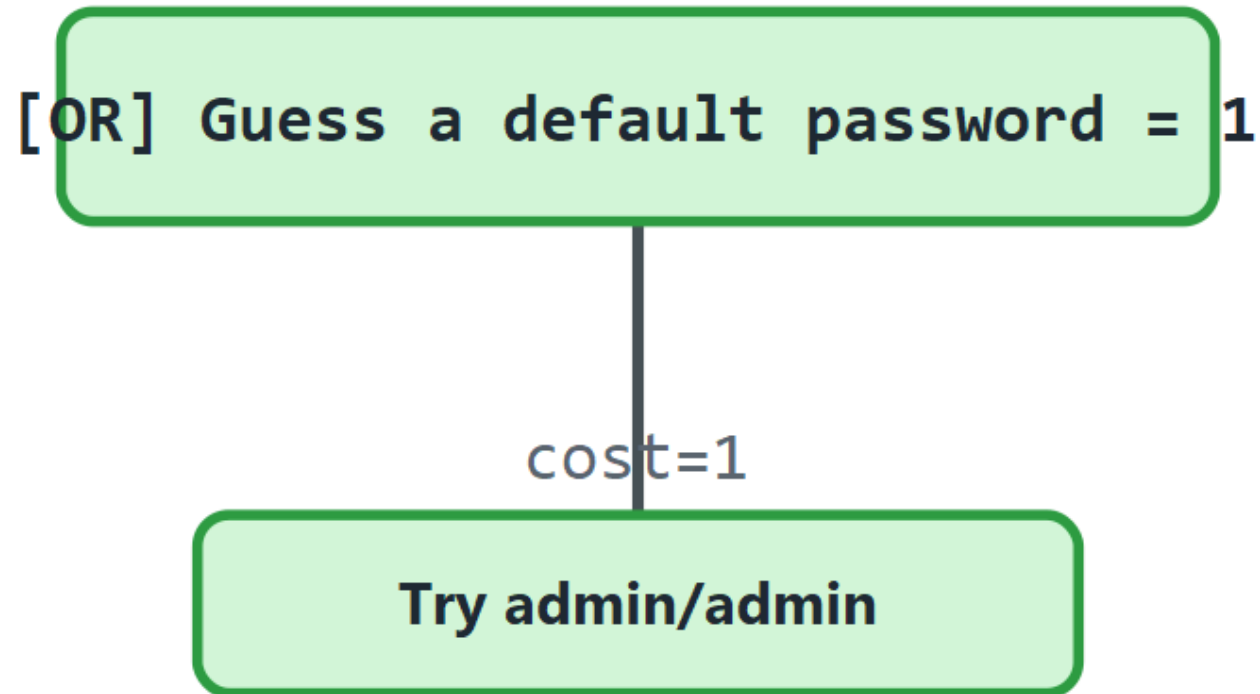
- "Bellekten oku" dalına **RASP** (anti-debug, anti-hook) eklendi.
- En ucuz saldırı **2 → 8** birime çıktı; en zayıf nokta **değişti**.
- Güvenlik yatırımını **sayıyla** gerekçelendirmenin yolu budur.

Saldırı ağacı: en ucuz yol (animasyon)



OR: kök = $\min(10, 5, 18) = 5$. EN UCUZ SALDIRI budur — savunmacının önce kırması gereken zincir.

Saldırı ağacı — uç durum: kök tek bir yaprak



OR: kök = $\min(1) = 1$. EN UCUZ SALDIRI budur — savunmacının önce kırması gereken zincir.

Yanılğı: "VE ile VEYA fark etmez"

Yanılğı: Saldırı ağacında düğüm türü önemli değil.

Gerçek: **VEYA** düğümünde en ucuz çocuk yeter; **VE** düğümünde çocukların **toplamı** gerekir.

Karıştırmak en ucuz yolu **yanlış buldurur** ve savunma parası **yanlış dala** gider.

Yanılgi: "En pahalı dalı güçlendirelim"

Yanılgi: Görünüşte en tehlikeli/karmaşık saldırı yolunu güçlendirmek en iyisidir.

Gerçek: Saldırgan her zaman **en ucuz** yolu seçer. Savunma köke giden **en ucuz yolu** pahalılaştırmıyorsa, en ucuz saldırının maliyeti hiç **değişmez** (Demo 04).

Yanılgi: "Önlem eklemek yeter"

Yanılgi: Ağaca bir savunma düğümü eklemek riski kapatır.

Gerçek: Her önlemin kendisi de **atlatılabilir**. Önlem düğümünün altına onu aşma yollarını (**karşı-karşı önlem**) yazmadan hesap **iyimser** çıkar.

Saldırı ağacı vs denetim kaydı: iki soru

- **Saldırı ağacı:** "Saldırgan **nereden** gelir?" — önceden, planlama sorusu.
- **Denetim kaydı (audit log):** "**Geldi mi**, ne yaptı?" — sonradan, kanıt sorusu.
- Bir olaydan sonra ne olduğunu anlamamanın, sorumluyu bulmanın tek yolu **güvenilir kayıttır**.
- Kitap bunu Tarif 13.11'de işler; öneriler bugün de geçerli, yalnız araçlar güncellendi.

Denetim kaydı: ne yazılır, ne yazılmaz?

✓ Kaydet	✗ Asla kaydetme
Giriş denemeleri (kim, ne zaman, nereden)	Parola, PIN, belirteç, özel anahtar
Yetki ve ayar değişiklikleri	Kart numarasının tamamı, kişisel veri
Hassas işlemler (onay, dışa aktarma, silme)	Çözülmüş gizli veri
Güvenlik denetimi sonuçları	Sürümde hata ayıklama çıktısı

Tarif 13.11 · günlüğü **saldırganın da okuyacağını** varsayarak tasarla

Denetim kaydı — şema

Güvenlik denetim kaydı: kanıt olarak günlük

olay olduktan sonra elinizde kalan tek şey

Ne yazılır?

kim, ne zaman,
ne yaptı, sonuç

Ne YAZILMAZ?

parola, anahtar,
kart no, kişisel veri

Nasıl korunur?

özet zinciri

kayıt 1

kayıt 2

kayıt 3

kayıt 4

Sondan kesme özet zinciriyle yakalanmaz; ayrıca uzak sunucuya kopyalayın.

Demo 10 — Günlük enjeksiyonu (CWE-117)

code/week-02/10-log-injection

UNSAFE:

```
AUDIT login user=bob
AUDIT login user=admin result=SUCCESS ... <- FORGED line
AUDIT login user=carol^[2K^Mhidden ... <- erases screen
```

SAFE:

```
AUDIT login user=bob\x0A2026-... (single line)
forged 'admin SUCCESS' line(s): 0
```

✓ Kaçışla (`\xNN`) · ✓ uzunluğu sınırla · ✓ `syslog(LOG_INFO, "%s", input)` — girdi asla biçim dizesi (format string) değil

Günlük enjeksiyonu: güvensiz vs güvenli (animasyon)

kullanıcı alanı (dosyadan ham bayt) =

b o b \n 2 0 2 6 - 0 9 - 2 5 0
9 : 1 5 : 4 2 A U D I T 1 o
g i n u s e r = a d m i n r
e s u l t = S U C C E S S

Sonuç: dosyada 2 satır oluştu (girilen tek alan yerine) — biri sahte, bağımsız bir kayıt.

Günlük enjeksiyonu — uç durum: ANSI kaçış dizisi

kullanıcı alanı (dosyadan ham bayt) =

c a r o l ESC [2 K \r h i d d e n
- a d m i n - t a k e o v e r -
a t t e m p t

`user=carol\x1B[2K\x0Dhidden-admin-takeover-attempt`

Sonuç: TEK satır garanti — kaçışlanmış metin, gönderenin kendi satırının İÇİNDE kalıyor.

Zafiyetli kod: günlük enjeksiyonu ve biçim dizesi

```
fprintf(log, "giris: %s\n", kullanıcı_adi); /* CWE-117 */
```

```
syslog(LOG_INFO, kullanıcı_girdisi); /* CWE-134 */
```

Kullanıcıdan gelen veri günlükte **kaçışsız satır sonu** ya da **doğrudan biçim dizesi** olarak kullanılmamalı:

```
syslog(LOG_INFO, "%s", girdi).
```

Özet zinciri: basit ama kırılabilir

- Fikir: her kayıt, **bir önceki kaydın özetini** içerir.
- Bir kaydı değiştirmek yalnız **o kaydın kendi özetini** bozar (doğrulayıcı her zaman saklanan zinciri kontrol eder, yeniden hesaplanmış değil).
- Sorun: özet **anahtarsız** — yetenekli bir saldırgan **sonraki tüm özetleri** de yeniden hesaplayıp kurcalamayı tamamen gizleyebilir.
- Çözüm: özeti **anahtarlı** yapmak → HMAC zinciri (bir sonraki slayt).

Sürüm derlemesinde günlük neden kaldırılır?

- Kayıt, bir **saldırganın da okuyabileceği** varsayılarak tasarlanır.
- Günlüğe yazılan her sır, günlük dosyasına erişen **herkese** verilmiş demektir.
- Kapalı bir bayrakla **susturulmuş** hata ayıklama kodu, binary dosyada **kalır**.
- Hem dizeleriyle (string) bilgi sızdırır hem de **yeniden açılabilir** — bu yüzden sürüm derlemesinde tamamen **çıkarılmalıdır**.

Demo 11 — Kurcalamaya dayanıklı günlük

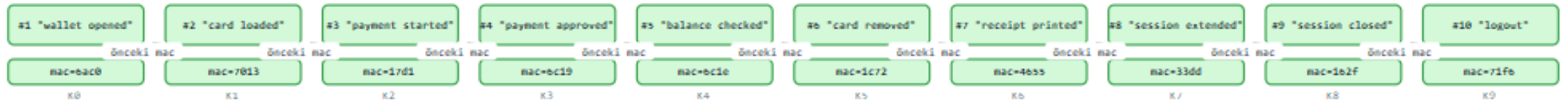
code/week-02/11-tamper-evident-log · HMAC zinciri + **anahtar evrimi**

Saldırı	Anahtar evrimli	Statik anahtar
Kayıt değiştirme	✓ MAC:BROKEN	✓ yakalanır
Kayıt silme / sıra değiştirme	✓ ORDER:BROKEN	✓ yakalanır
Ele geçirilen anahtarla geçmişi yeniden yazma	✓ TAMPERED	✗ INTACT görünür

Anahtar her kayıta tek yönlü evrilir, eski anahtar silinir → ele geçirmeden **önceki** kayıtlar korunur

Kurcalamaya dayanıklı günlük: HMAC zinciri (animasyon)

ANAHTAR EVRİMİ: her kaydın mac'ı öncekinin mac'ini de kapsar



Doğrulama: her kayıt için mac yeniden hesaplanıyor, hepsi saklanan değerle eşleşiyor — zincir SAĞLAM.

HMAC zinciri — uç durum: statik anahtar (evrim yok)

SABİT ANAHTAR: her kaydın mac'i öncekinin mac'ini de kapsar



>>> SONUÇ: günlük KURCALANMIŞ (doğrulama başarısız).

İleriye güvenlik: anahtar neden evrilir?

- Her kayıttan sonra anahtar **tek yönlü** bir fonksiyonla değişir; eski anahtar **silinir**.
- Saldırgan makineyi ele geçirdiğinde yalnız **o anki** anahtarı bulur.
- O andan **geriye**, önceki kayıtları mühürleyen anahtarlara **dönemez**.
- Sonuç: ele geçirilmeden **önceki** kayıtlar korunur (Schneier–Kelsey, 1999).

Soru — sondan kesme yakalanır mı?

Demo 11'in ürettiği zincirden **son üç satırı silin**.

Soru: Doğrulayıcı bunu fark eder mi? Neden?

Cevap: Yalnızca zincirin **son değeri** ya da **kayıt sayısı** ayrıca (uzak bir sunucuda) tutuluyorsa fark eder. Zincirin kendisi yalnız dosyada tutulursa, sondan kesme **iç tutarlılığı bozmaz** — çünkü silinen kayıtlardan sonra gelen hiçbir kayıt yoktur ki tutarsızlık gösterebilir.

Kural (özet): denetim kaydı

- Kullanıcı verisini **kaçışlayarak ve sınırlandırarak** yazın.
- Sırları (parola, anahtar) **asla** yazmayın.
- Günlüğü **anahtarlı bir zincirle** mühürleyin; anahtarı her kayıta **evriltin**.
- Doğrulama anahtarını günlüğün bulunduğu **makinede tutmayın**.
- Mümkünse kayıtları **anında** ayrı bir sunucuya gönderin.

Sahada nasıl uygulanır? — mobil günlük

- Mobil uygulamalarda **cihaz üzerindeki** günlük en az tutulur, hassas alan içermez.
- Güvenlik olayları (bütünlük hatası, hata ayıklayıcı algılama) **sunucuya raporlanır**.
- Bu, 6. haftada göreceğimiz **RASP** önlemlerinin "raporlama" ayağıdır.
- Cihazdaki günlük saldırganın elindedir; asıl kanıt **uzaktaki** kopyadır.

Yanılgı: "Denetim kaydına her şeyi yazalım"

Yanılgı: Daha çok bilgi, daha iyi kanıt demektir.

Gerçek: Parola, PIN, anahtar ve **tam kart numarası** kayda **asla** girmez (CWE-532).

Kayıt, olayı yeniden kurmaya **yetecek kadar** bilgi taşır: kim, ne, ne zaman, nerede, sonuç.

Yanılğı: "Kullanıcı adını olduğu gibi yazdım"

Yanılğı: Kullanıcıdan gelen alan doğrudan günlüğe yazılabilir.

Gerçek: İçinde satır sonu olan bir girdi **sahte kayıt satırı** üretir (CWE-117, Demo 10).

Denetim karakterleri **kaçışlanmalı** ya da yapılandırılmış (alan alan) kayıt kullanılmalıdır.

Yanılgı: "Kayıt sonuna özet ekledim, kurcalama anlaşılır"

Yanılgı: Dosyanın sonuna bir özet eklemek yeterli korumadır.

Gerçek: Anahtarsız özeti saldırgan **yeniden hesaplar**.

HMAC zinciri araya ekleme/değiştirmeyi yakalar; **sondan kesmeyi** yakalamak için son zincir değeri ayrıca (**uzak sunucuda**) tutulmalıdır (Demo 11).

Kontrol listesi — saldırı ağacı ve denetim kaydı

- [] Bir saldırı ağacını **VE/VEYA** kurallarıyla aşağıdan yukarı çözüp en ucuz yolu bulabiliyorum.
- [] Bir savunmanın en ucuz yolu **nasıl değiştirdiğini** sayıyla gösterebiliyorum (Demo 04).
- [] Bir denetim kaydı satırında **olması ve olmaması** gerekenleri sayabiliyorum.
- [] Günlük enjeksiyonunun (CWE-117) nasıl önlendiğini açıklayabiliyorum.
- [] HMAC zincirinin ve anahtar evriminin neyi yakalayıp **neyi yakalayamadığını** söyleyebiliyorum.

4. Erişim denetimi ve modeller

Özne, nesne, hak — ve matris

- **Özne** (kullanıcı/süreç) · **Nesne** (dosya/kayıt) · **Hak** (oku/yaz/çalıştır).

	payment_key	log
uygulama	—	yaz
kutuphane	oku, yaz	yaz
saldirgan	—	—

Matris iki biçimde saklanır: **ACL** (sütun: nesne kimin erişeceğini tutar) · **Yetenek** (satır: özne biletleri).

DAC · MAC · RBAC

Model	Kararı kim verir?	Örnek
DAC	Nesnenin sahibi	Unix <code>chmod</code> , Windows DACL
MAC	Sistem/politika (sahip değiştiremez)	Bell–LaPadula, SELinux
RBAC	İzinler rollere , kullanıcı role atanır	"Muhasebeci" rolü

Gerçek sistemler üçünü **birlikte** kullanır; etkin karar çoğu zaman **kesişimdir**.

DAC **esnek** ama sızıntıya açık; MAC **kati** (askeri/etiketli); RBAC **kurumsal**, rol bazlı ve yönetimi kolaydır.

DAC ve MAC — şema

Erişim denetimi: kim, neye, ne yapabilir?

iki ana model, iki farklı güven varsayımı

İSTEĞE BAĞLI (DAC)

sahibi izinleri belirler
Unix rwx, Windows ACL
esnek
kullanıcı yanlış izin verebilir

ZORUNLU (MAC)

politika merkezî
etiket/seviye ile karar
kullanıcı gevşetemez
kurulumu zor

RBAC ikisinin arasındadır: yetkiyi kişiye değil **ROLE** verir.

Demo 03 — Salt DAC (matris)

code/week-02/03-access-model

```
library(D ) R payment_key (D ) DAC:YES => ALLOW  
app      (D ) R payment_key (D ) DAC:no  => DENY (matriste yok)  
attacker(D ) R payment_key (D ) DAC:no  => DENY
```

Uygulamanın bile ödeme anahtarına **doğrudan erişimi yok** → **en az ayrıcalık**.

Bell–LaPadula: gizlilik

Seviyeler: Genel < Gizli < Çok Gizli

- **Yukarı okuma yok:** kendinden gizli olanı okuyamaz.
- **Aşağı yazma yok (*-özellik):** kendinden az gizli olana yazamaz.

"Read down, write up" — gizli bilgi **aşağı sızamaz**.

Örnek: "Gizli" etiketli bir kullanıcı "Çok Gizli" bir belgeyi **okuyamaz**; "Çok Gizli" bir süreç "Genel" bir dosyaya **yazamaz** (sızıntı böyle önlenir).

Biba: bütünlük (BLP'nin tersi)

Seviyeler: Dış < Uygulama < Çekirdek (güvenilirlik)

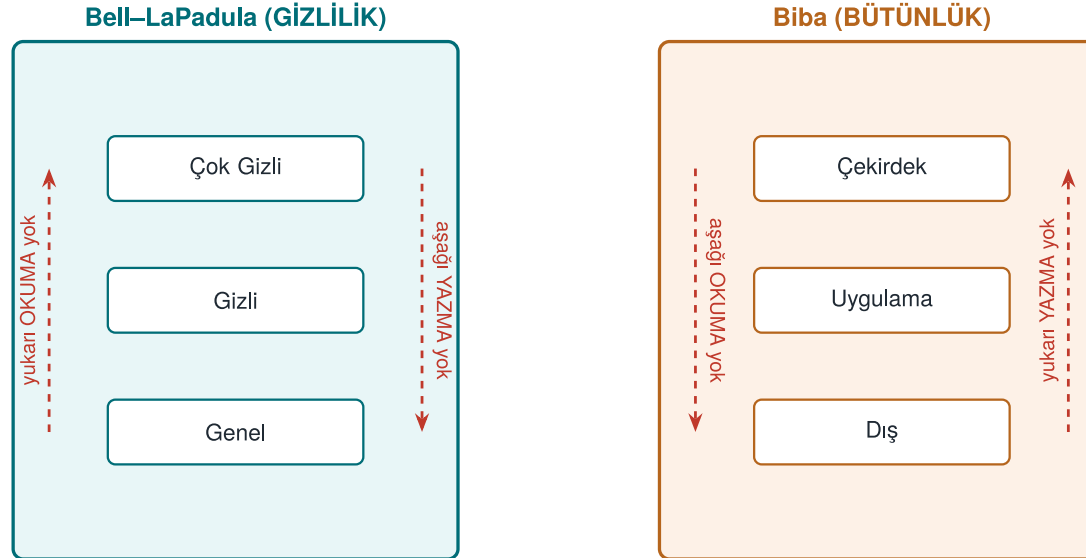
- **Aşağı okuma yok:** kendinden az güvenilir veriyi okuyamaz.
- **Yukarı yazma yok:** kendinden güvenilir veriye yazamaz.

"Read up, write down" — kirli bilgi **yukarı çıkamaz**, temizi bozamaz.

BLP ve Biba: ters yönler

Bell–LaPadula ↔ Biba: ters yönler

BLP gizliliği, Biba bütünlüğü korur



Aynı kafes, ters kurallar: biri sırrın aşağı sızmasını, diğeri çöpün yukarı çıkmasını engeller.

Ters yönler: BLP gizliliği (yukarı okuma + aşağı yazma yok), Biba bütünlüğü (aşağı okuma + yukarı yazma yok) korur.

Bir veri hem gizli hem güvenilir olmalıysa → yalnız **kendi seviyesinde** işlenir.

Clark–Wilson: ticari bütünlük

Askeri seviye yerine **iyi biçimli işlem + görev ayrılığı** (1987, banka):

- **CDI**: bütünlüğü korunacak veri (bakiye).
- **TP**: CDI'ye yalnız **onaylı işlemler** dokunur (doğrudan yazma yok).
- **IVP**: tutarlılığı düzenli denetler ("borç = alacak").
- **Görev ayrılığı**: başlatan $\neq$ onaylayan.

"Veriye doğrudan değil, **doğrulanmış işlemlerle** dokun."

Demo 03 — BLP ve Biba çıktısı

```
BLP: clerk    READ  operation  BLP:DENY => DENY  (read up)
     general  WRITE notice     BLP:DENY => DENY  (write down)

Biba: listener WRITE record    BIBA:DENY => DENY  (write up)
     processor READ  incoming  BIBA:DENY => DENY  (read down)
```

DAC her ikisinde **YES**; reddi getiren **sistem (MAC)** kuralıdır.

Erişim matrisi: DAC + BLP + Biba + Chinese Wall (animasyon)

BLP (Gizlilik) — istek: READ

S: level=0 {EUROPE}

O: level=2 {EUROPE}

=> REDDEDİLDİ: etiket ilişkisi kurala uymuyor.

Erişim matrisi — uç durum: eşit etiketler

BLP (Gizlilik) — istek: WRITE

S: level=1 {ASIA}

O: level=1 {ASIA}

=> İZİN VERİLDİ: etiket ilişkisi kurala uyuyor.

Unix erişim modeli

`-rwxr-x---` → sahip / grup / diğerleri, her biri **oku-yaz-çalıştır**.

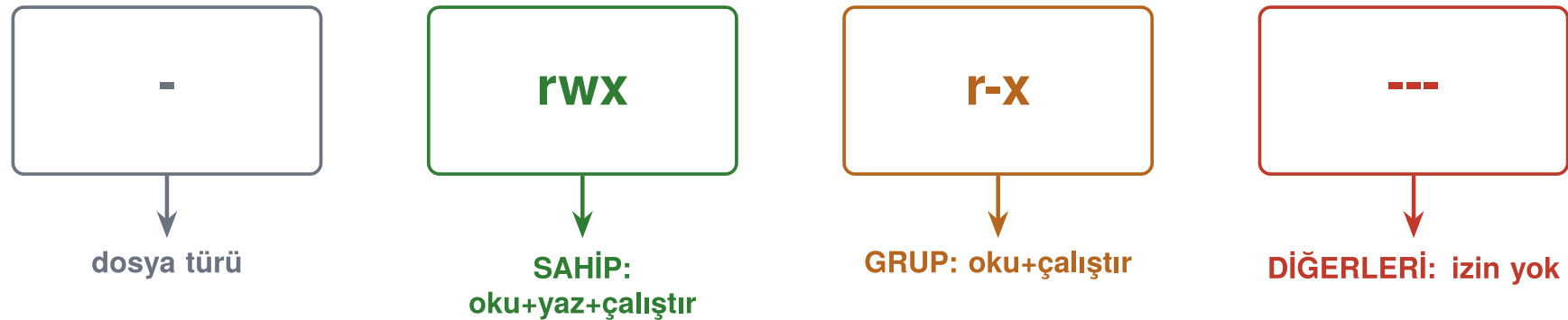
- Üç kimlik: **effective / real / saved** UID.
- **setuid**: program **sahibinin** (çoğu zaman root) yetkisiyle çalışır — güçlü ama tehlikeli.
- **sticky bit**: `/tmp` 'te başkasının dosyasını silememe.

Kitap: Tarif 2.1

Unix izin bitleri — şema

Unix izin bitleri: -rwxr-x---

üç grup, üçer bit



Her grup 3 bit: $r=4, w=2, x=1 \rightarrow rwx = 7, r-x = 5, --- = 0 \Rightarrow \text{chmod } 750$

En az ayrıcalık: “diğerleri” sütununu boş bırakmak çoğu sızıntıyı baştan keser.

Windows erişim modeli

Kavram	Unix	Windows
Kimlik	UID/GID	SID
İzin	Mod bitleri	DACL içindeki ACE'ler
Denetim	syslog (harici)	SACL (yerleşik)
"Herkes açık"	chmod 777	NULL DACL

- **DACL**: kim ne yapabilir (ACE = SID + hak + izin/reddet).
- Modern Windows: **MIC** ve **AppContainer** (Biba benzeri). Kitap: Tarif 2.2

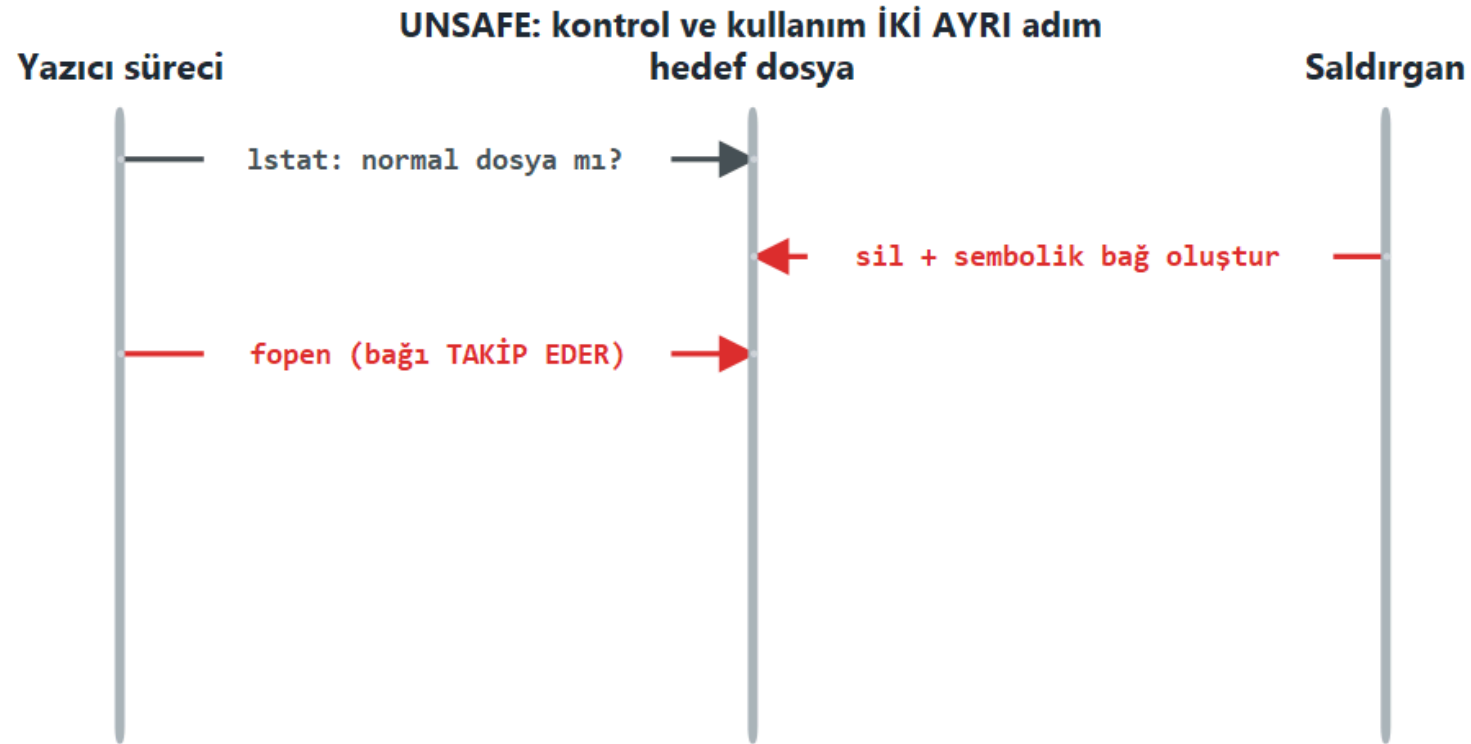
Demo 06 — TOCTOU yarışı (CWE-367)

code/week-02/06-toctou (yalnız WSL/Linux) · Kitap: Tarif 2.3

```
UNSAFE (lstat first, then fopen):  
  >>> ATTACK SUCCEEDED: write redirected to secret_target.txt!  
SAFE (O_NOFOLLOW, one step):  
  rejected: the target is a symbolic link
```

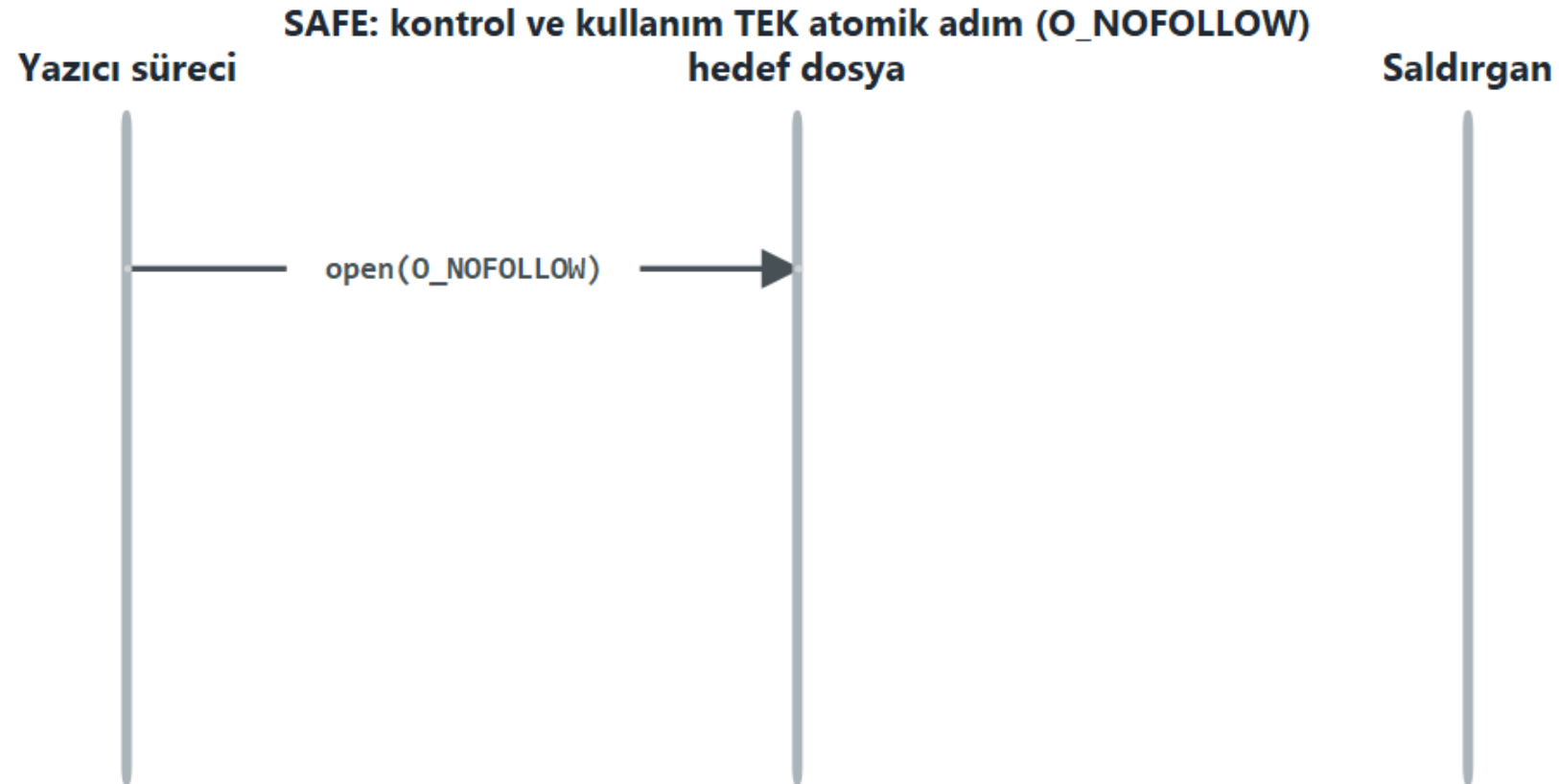
Denetim ile kullanım arasındaki boşluk bir yarıştı. Çözüm: tek atomik adım (`O_NOFOLLOW` , `O_EXCL`), fd üzerinden çalış.

TOCTOU yarışı: denetim ile kullanım arasındaki pencere (animasyon)



>>> SALDIRI BAŞARILI: yazma secret_target'a yönlendirildi.

TOCTOU — uç durum: **SAFE**, saldırgan yok



>>> Saldırgan araya girmedi: yazma normal şekilde tamamlandı.

Demo 12 — ACE değerlendirme algoritması

code/week-02/12-ace-evaluation · Tarif 2.2

```
1a canonical: DENY Everyone FULL | ALLOW Marketing WRITE => DENY
1b broken:    ALLOW Marketing WRITE | ... DENY Everyone   => ALLOW
2a NULL DACL (unrelated account) => ALLOW (TAKE_OWNER, WRITE_DAC dahil!)
2b EMPTY DACL (same account)     => DENY
3b inheritance broken            => DENY (list ended)
```

- Sıra önemli: **açık DENY** → **açık ALLOW** → **miras DENY** → **miras ALLOW**
- ❌ NULL DACL = herkese **her** hak · ✅ gerekli hakları açıkça ver
- Sahip, DACL ne derse desin izinleri değiştirebilir

Windows ACE değerlendirme sırası (animasyon)

Token: alice gruplar: Users

İstek: READ,WRITE

1. ALLOW alice READ

2. ALLOW Users WRITE

İstenen her hak verildi => İZİN VERİLDİ.

ACE sırası — uç durum: kurallı sıra, DENY ilk ACE

Token: carl gruplar: Everyone

İstek: DELETE

1. DENY Everyone DELETE

2. ALLOW car1 READ,WRITE,DELETE

=> REDDEDİLDİ.

Demo 13 — Unix izinleri, umask, setuid

code/week-02/13-unix-permissions-umask · Tarif 2.1, 2.7, 1.3

```
alice read paradox.txt ---rwxrwx owner -> DENY
zoe   read paradox.txt ---rwxrwx other -> ALLOW
0666 & ~umask: 000 -> rw-rw-rw- 022 -> rw-r--r-- 077 -> rw-----
seteuid(1001) r/e/s = 1001/1001/0 <- saved UID still 0!
```

- **İlk eşleşen sınıf** karar verir (sahip → grup → diğerleri)
- `fopen()` 0666 ister → hassas dosya: `open(..., 0600)`
- Kalıcı bırakma: `setresuid()` + doğrula

Unix izin denetimi ve umask (animasyon)

İstek: READ

`uid=1001 groups=[2001]`

`owner=1001 group=2001 mode=rw-r-----`

`bits(110) & READ(r) != 0 -> İZİN VERİLDİ.`

Unix izinleri — uç durum: umask 000 tehlikesi

gerçek izin = istenen & ~umask

	r	w	x	r	w	x	r	w	x
istenen	1	1	0	1	1	0	1	1	0
~umask	1	1	1	1	1	1	1	1	1
sonuç	1	1	0	1	1	0	1	1	0

gerçek izin = rw-rw-rw- (666)

Sonuç: rw-rw-rw-. umask 000 ile 0666 isteyince dosya HERKESE YAZILABİLİR olur — dikkat.

Demo 14 — RBAC, görev ayrılığı, Clark–Wilson

code/week-02/14-rbac-clark-wilson · banka benzetimi

```
ASSIGN carl TELLER          SSD(TELLER,APPROVER) -> DENY
SESSION dana TELLER,AUDITOR DSD(TELLER,AUDITOR) -> DENY
SESSION dana AUDITOR        -> OK
dana IVP: expected 20250, found 20250
```

- **SSD**: çelişen iki role atanamaz · **DSD**: aynı oturumda ikisi etkin olamaz
- CDI'ya yalnız **sertifikalı TP** dokunur; büyük havale → **ikinci kişi onayı**
- ⚠ Hatalı TP sertifikalanırsa kurallar görmez, **IVP yakalar**

RBAC + Clark–Wilson: işlem denetim kapıları (animasyon)

alice -> DEPOSIT çalıştırmak istiyor

E3: kimlik tanınıyor mu?

E3: açık oturum var mı?

E4: sertifikacı değil mi?

E2: TP, aktif bir rolde mi?

E1: TP bu CDI'lar için sertifikalı mı?

5 kapı da geçti => TP ÇALIŞABİLİR (C2/C3/C5 kontrolleri TP'nin kendi içinde devam eder).

RBAC — uç durum: kullanıcı hiç tanınmıyor

guest -> DEPOSIT çalıştırmak istiyor

E3: kimlik tanınıyor mu?

E3: açık oturum var mı?

E4: sertifikacı değil mi?

E2: TP, aktif bir rolde mi?

E1: TP bu CDI'lar için sertifikalı mı?

=> REDDEDİLDİ (kapı: E3A).

Yanılgı: "Kimliği doğruladık, yetkiyi de doğrulamış olduk"

Yanılgı: Giriş yapabilen kullanıcı, her şeyi yapabilir.

Gerçek: Kimlik doğrulama "sen kimsin?" sorusudur; **yetkilendirme** "bunu yapabilir misin?" sorusudur — ayrı adımlardır.

2025 CWE Top 25'te dördüncü sıradaki **CWE-862** (eksik yetkilendirme) tam bu hatadır.

Yanılğı: "Hata olursa izin verelim"

Yanılğı: Politika okunamıyorsa ya da belirsizse, kullanıcı mağdur olmasın diye **izin verilmeli**.

Gerçek: Güvenli varsayılan ilkesi tam tersini söyler: politika okunamıyorsa cevap **ret** olmalıdır (CWE-636, "fail-open" hatası).

"Fail-closed" (hata olunca kapat) güvenli sistemlerin varsayılanıdır.

Yanılgı: "NULL DACL ile boş DACL aynı"

Yanılgı: İkisi de "izin yok" gibi görünür.

Gerçek: **NULL DACL** (hiç DACL yok) **herkese tam erişim** verir — sahiplik alma dahil. **Boş DACL** (ACE'siz liste) ise **kimseye hiçbir hak vermez**.

İlki ciddi bir zafiyettir (CWE-732); ikisini karıştırmak tehlikelidir (Demo 12).

Yanılgı: "ACE sırası önemli değil"

Yanılgı: Aynı ACE'ler farklı sırada yazılsa da sonuç aynıdır.

Gerçek: Windows ACE'leri **sırayla** değerlendirir ve **ilk belirleyici** girdide durur.

Kanonik sırada açık ret girdileri izinlerden önce gelir; elle yanlış sıralanmış bir DACL, niyet edilen reddi **etkisiz bırakabilir** (Demo 12).

Yanılgı: "chmod 777 ile sorun çözüldü"

Yanılgı: "Erişim reddedildi" hatasını `chmod 777` ile geçiştirmek pratik bir çözümdür.

Gerçek: Sorun izinlerin herkese açılmasıyla **çözülmez, büyütülür**.

Sırlar `0600` ile oluşturulmalı, `umask` programın başında **bilinçli** ayarlanmalıdır (Demo 13).

Yanılğı: "RBAC kurduk, görev ayrılığı kendiliğinden gelir"

Yanılğı: Rol tabanlı erişim denetimi kurmak, görev ayrılığını otomatik sağlar.

Gerçek: Aynı kişiye hem "iade başlat" hem "iade onayla" rolü verilirse RBAC görev ayrılığını sağlamaz.

Statik (**SSD**) ya da dinamik (**DSD**) görev ayrılığı kısıtı **ayrıca** tanımlanmalıdır (Demo 14).

Mobil izin modeli ve en az ayrıcalık

- Her uygulama kendi **kum havuzunda** çalışır — diğer uygulamaların verisine varsayılan olarak erişemez.
- Hassas kaynaklar (kamera, konum, kişiler) **çalışma anı izni** ister; kullanıcı onaylamadan erişilmez.
- Bu, DAC (kullanıcı onayı) ile MAC'e (sistemin zorunlu kum havuzu sınırı) benzer bir **karma modeldir**.
- En az ayrıcalık ilkesinin (Hafta 1) günümüz mobil işletim sistemlerindeki **somut hâlidir**.

Kontrol listesi — erişim denetimi

- [] DAC, MAC ve RBAC'ı "**kararı kim verir?**" sorusuyla ayırabiliyorum.
- [] BLP, Biba ve Clark–Wilson kurallarını bir senaryoya uygulayabiliyorum.
- [] Kimlik doğrulama ile yetkilendirme **farkını** bir örnekle açıklayabiliyorum.
- [] `ls -l` çıktısını, setuid/sticky bitlerini ve umask hesabını yorumlayabiliyorum.
- [] NULL DACL, boş DACL ve ACE sırasının neden **kritik** olduğunu gösterebiliyorum.
- [] RBAC'ta SSD/DSD ile görev ayrılığının nasıl **ayrıca** kurulduğunu açıklayabiliyorum.

5. Zafiyet sınıflandırma

CWE: zayıflık türleri kataloğu

- **CWE** = zayıflığın **türü** (SQL enjeksiyonu → CWE-89).
- **CWE Top 25**: o yıl en çok görülen/tehlikeli 25 zayıflık.

CWE	Zayıflık
CWE-787	Sınır dışına yazma
CWE-79	XSS
CWE-89	SQL enjeksiyonu
CWE-367	TOCTOU (Demo 06)
CWE-798	Gömülü kimlik/anahtar

Hiyerarşik: pillar → class → base → variant. **En somut** CWE'yi seç.

Yanılgı: "CWE ile CVE aynı şey"

Yanılgı: İkisi de aynı anlama gelir, birbirinin yerine kullanılabilir.

Gerçek: **CWE** zayıflığın **türüdür** (SQL enjeksiyonu → CWE-89). **CVE** belirli bir ürünlerdeki belirli açığın **kimliğidir** (Heartbleed → CVE-2014-0160).

Bir CVE, bir ya da birden çok CWE türüne **aittir**.

Yanılğı: "En genel CWE'yi seçmek güvenlidir"

Yanılğı: Emin olmadığında en üst düzey (genel) CWE'yi yazmak daha güvenlidir.

Gerçek: CWE-20 (girdi doğrulama) gibi üst düzey girdiler **düzeltilmeyi anlatmaz**.

Kural: mümkün olan **en somut** (tercihen base düzeyi) CWE'yi seçin.

OWASP Top 10 ve MASVS

Belge	Kapsam	Türü
CWE / Top 25	Tüm yazılım	Katalog / öncelik
OWASP Top 10	Web	Farkındalık / öncelik
OWASP ASVS	Web	Doğrulama standardı
OWASP MASVS + MASTG	Mobil	Doğrulama + test

MASVS-RESILIENCE = tersine mühendisliğe direnç → bu dersin kod gizleme (H9, H14), RASP (H6), whitebox (H11) konuları.

OWASP ve MASVS — şema

OWASP Top 10 ve MASVS: ne işe yararlar?

liste ile gereksinim aynı şey değildir

OWASP Top 10

en yaygın web açıkları — farkındalık listesi, denetim listesi DEĞİL

OWASP ASVS

doğrulama gereksinimleri — seviyelere ayrılmış, denetlenebilir

OWASP MASVS

mobil uygulama güvenlik gereksinimleri — bu dersin projesine en yakını

MASTG

MASVS'i nasıl test edeceğinizi anlatan rehber

13. haftada uyum matrisini MASVS üzerinden kuracağız.

Yanılgı: "OWASP Top 10 bir kontrol listesidir"

Yanılgı: Top 10'daki 10 maddeyi karşılıyorsam güvenliyim demektir.

Gerçek: Top 10 bir **farkındalık** belgesidir, doğrulama standardı değil.

Doğrulanabilir gereksinimler web için **ASVS'de**, mobil için **MASVS ve MASTG'dedir**.

CVE ve CVSS

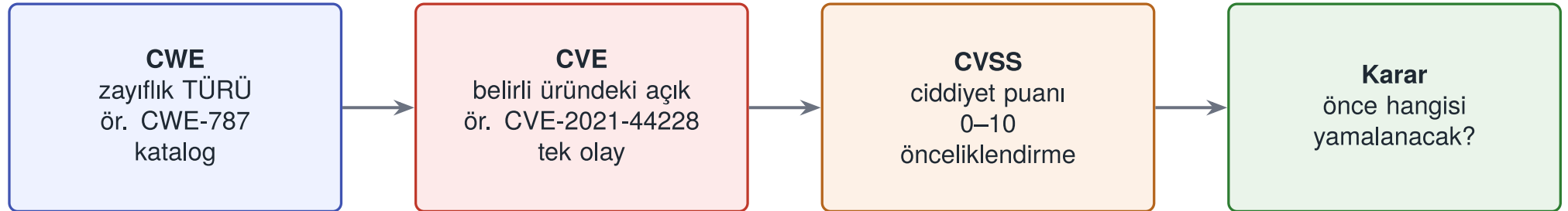
- **CVE:** belirli ürünlerdeki belirli açığın **kimliği** (CVE-2014-0160 = Heartbleed). Bir **isim**, ciddiyet değil.
- **CVSS:** açığın **ciddiyeti** 0.0–10.0. Üç grup: **Temel** (değişmez), Zamansal, Çevresel.

Bir CVE bir ya da birden çok **CWE türüne** aittir.

CWE, CVE, CVSS — şema

CWE, CVE, CVSS: üç ayrı soru

tür mü, olay mı, ne kadar ciddi mi?



Karıştırmayın: CWE sınıf, CVE örnek, CVSS ölçü.

CVSS v3.1 Temel puan metrikleri

Metrik	Sorusu	Değerler
AV	Nereden?	Ağ / Bitişik / Yerel / Fiziksel
AC	Ne kadar zor?	Düşük / Yüksek
PR	Yetki gerekir mi?	Yok / Düşük / Yüksek
UI	Kurban gerekir mi?	Yok / Gerekli
S	Kapsam aşıyor mu?	Değişmedi / Değişti
C/I/A	Etki?	Yüksek / Düşük / Yok

v4.0: kapsam yerine ayrı VC/VI/VA + SC/SI/SA; temel puan = CVSS-B.

Demo 05 — CVSS hesaplayıcı

code/week-02/05-cvss (Python)

```
AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H = 9.8 Kritik
... S:U -> S:C = 10.0 Kritik
AV:L (yere1) /PR:L/... C:H/I:H/A:H = 7.8 Yüksek
```

- **Uzaktan + kimliksiz** en yüksek; kapsam değişince 10.0.
- ⚠ Temel puan **bağlamı bilmez**: 4.0'lık açık tam sizin varlığını vuruyorsa sizin için 9.8'den önemli olabilir.

CVSS v3.1 taban puan hesabı (animasyon)

Saldırı vektörü	AV:N (Network)	0.85	Kapsam	S:U (Unchanged)	
Saldırı karmaşıklığı	AC:L (Low)	0.77	Gizlilik etkisi	C:H (High)	0.56
Gereken yetki	PR:N (None)	0.85	Bütünlük etkisi	I:H (High)	0.56
Kullanıcı etkileşimi	UI:N (None)	0.85	Erişilebilirlik etkisi	A:H (High)	0.56

PR table = PR_U[N] = 0.85

ISS = $1 - (1-0.56)(1-0.56)(1-0.56) = 0.915$

Exploit. = $8.22 * 0.85 * 0.77 * 0.85 * 0.85 = 3.89$

Impact = 5.873

Base = 9.7602

BASE SCORE = roundup(9.7602) = 9.8

"roundup" NORMAL yuvarlama değildir — her zaman YUKARI yuvarlar (7.21 -> 7.3, asla 7.2 değil).

CVSS — uç durum: hiç etki yok

Saldırı vektörü	AV:N (Network)	0.85	Kapsam	S:U (Unchanged)	
Saldırı karmaşıklığı	AC:L (Low)	0.77	Gizlilik etkisi	C:N (None)	0.00
Gereken yetki	PR:N (None)	0.85	Bütünlük etkisi	I:N (None)	0.00
Kullanıcı etkileşimi	UI:N (None)	0.85	Erişilebilirlik etkisi	A:N (None)	0.00

$$\text{PR table} = \text{PR_U}[N] = 0.85$$

$$\text{ISS} = 1 - (1-0.00)(1-0.00)(1-0.00) = 0.000$$

$$\text{Impact} = 0.000$$

Impact ≤ 0 : hiçbir güvenlik özelliği bozulmuyor — puan doğrudan 0.0.

CVSS puanı → ciddiye bandı

Puan	Bant
0.0	Yok
0.1–3.9	Düşük
4.0–6.9	Orta
7.0–8.9	Yüksek
9.0–10.0	Kritik

Aynı etki (C:H/I:H/A:H) olsa bile **uzaktan + kimliksiz** açık, **yerel + yetkili** açıktan daha yüksek bant alır.

CVSS v4.0 kısaca

- 2023'te geldi; v3.1'in bazı zayıflıklarını giderir.
- Temel puan artık **CVSS-B** olarak anılır.
- **Kapsam (S)** metriği kaldırıldı; yerine ayrı **Zafiyet Etkisi** (VC/VI/VA) ve **Sonraki Sistem Etkisi** (SC/SI/SA).
- Sömürü olgunluğu (E) ve tehdit/çevre metrikleriyle **CVSS-BTE** hesaplanabiliyor.
- Bu derste hesap makinesi v3.1 için yazıldı; sahada hâlâ en yaygın odur.

Yanılgi: "v3.1 vektörünü v4.0'a girerim"

Yanılgi: CVSS v3.1 vektörü, v4.0 hesap makinesinde de doğrudan çalışır.

Gerçek: v4.0'da **Kapsam (S)** yoktur; **AT, VC/VI/VA, SC/SI/SA** metrikleri vardır ve UI üç değer alır (N/P/A).

Vektörler birbirine **çevrilmez**, yeniden **kurulur**.

CVSS'in sık yapılan hatası

Yanılğı: "CVSS 9.8, hemen yamalayalım; 4.0, beklesin."

- Temel puan **bağlamı bilmez**: açığın kendi özelliklerini ölçer.
- 4.0'lık bir açık, tam sizin **en değerli varlığınızın** kapısıysa sizin için 9.8'den önemli olabilir.
- Bunun için **Çevresel** metrikler ve **varlık tablonuz** (S5) vardır.
- Puan önceliklendirmeye **başlangıçtır**, son söz değildir.

Değerlendirici: bulgudan puana

Bir bulguyu **tekrarlanabilir** biçimde derecelendirmenin somut soruları:

- Saldırgan bunu **ağdan** mı, yoksa yalnız **cihaz elindeyken** mi kullanabiliyor (AV)?
- Önceden bir **yetki** ya da **kullanıcı etkileşimi** gerekiyor mu (PR, UI)?
- Etki açığın olduğu bileşende mi kalıyor, dışına mı taşıyor (S)?

Bu dersin bağlamında (program saldırganının cihazında) çoğu bulgu **yerel (AV:L)** çıkar; puan düşse de risk düşmez.

CVSS'in ötesi: EPSS

- **EPSS** (Exploit Prediction Scoring System): bir açığın **30 gün içinde istismar edilme olasılığı** (0–1 arası).
- **CVSS ciddiyeti** ölçer, **EPSS olasılığı** ölçer — ikisi **farklı sorulara** cevap verir.
- "EPSS yüksek, o hâlde CVSS de yüksektir" **yanlıştır**; ikisi birbirinden bağımsız ölçülür.
- İkisi **birlikte** okunduğunda öncelik sırası netleşir.

CVSS'in ötesi: KEV

- **KEV** (Known Exploited Vulnerabilities): **kanıtlanmış** biçimde istismar edilen açıkların kataloğu.
- Her kayıt: CVE kimliği, istismar kanıtı, açık düzeltme yönergesi.
- "KEV'de yoksa istismar edilmüyordur" **yanlıştır** — listede olmamak güvenlik kanıtı değildir.
- KEV'deki bir açık, düşük CVSS puanı olsa bile **öncelikli** ele alınmalıdır.

SSVC: karar noktaları

Bir bulguyu **karara** çeviren beş soru:

- İstismar var mı? · Otomatikleştirilebilir mi?
- Teknik etki nedir? · Görev ne kadar yaygın etkilenir?
- Kamu esenliği nasıl etkilenir?

Sonuç dört karardan biri: **Track / Track* / Attend / Act**

Örnek: beş bulgu, iki sıralama

No	Bulgu	Sistem	EPSS	KEV
B1	Ayırıştırıcıda uzunluk denetimi yok, uzaktan kod çalıştırma	İnternete açık	0,62	Evet
B5	Sürücüde yerel yetki yükseltme	Çalışan bilgisayarları	0,41	Evet
B4	Çerezde Secure bayrağı yok	İnternete açık	0,08	Hayır

Yalnız CVSS'e göre sıralama ile EPSS+KEV eklenmiş sıralama **farklı** çıkabilir; B5, CVSS'te ortalarda kalsa da KEV'de olduğu için **öne çıkar**.

Yanılğı: "Cihazdaki açıklar düşük puan alıyor, önemsiz"

Yanılğı: AV:L (yerel erişim) çıkan bir açık, düşük puanlıysa göz ardı edilebilir.

Gerçek: Bu dersin saldırgan modelinde **cihaz zaten saldırganın elindedir** (beyaz kutu).

Puan düşse bile risk **azalmaz**; puanın yanına **saldırı potansiyeli** (süre, uzmanlık, ekipman) de yazılmalıdır.

Pratik: vektörü siz kurun

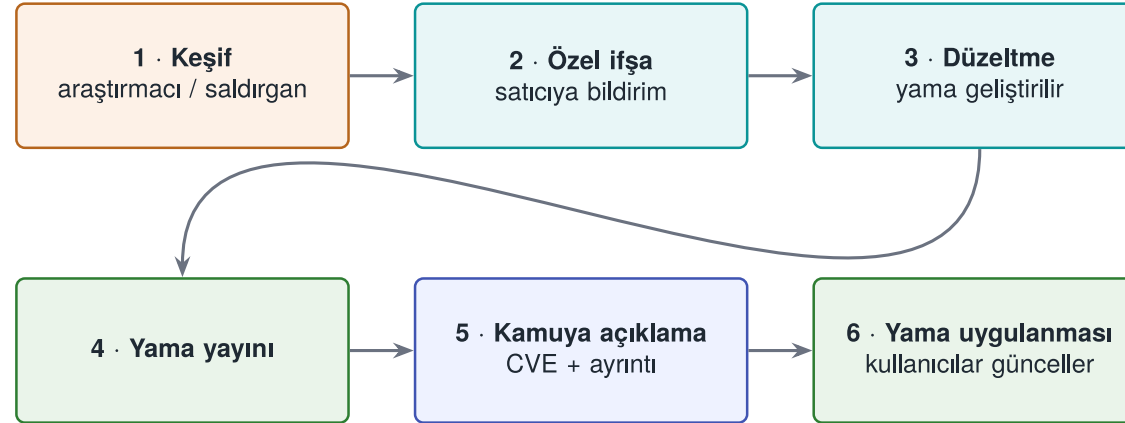
Soru: Bir saldırgan **ağdan**, **kimlik doğrulamadan**, **kullanıcı etkileşimi olmadan** bir sistemi tamamen ele geçiriyor (gizlilik, bütünlük, erişilebilirlik hepsi tam bozuluyor). CVSS v3.1 vektörü ve puanı nedir?

Cevap: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H → **9.8 (Kritik)**. Kapsam da değişseydi (S:C) puan 10.0'a çıkardı.

Zafiyet yaşam döngüsü

Zafiyet yaşam döngüsü ve sorumlu ifşa

doğru yol: önce satıcıya bildir, yama çıksın, sonra açıkla



Kötüye giderse: yama yokken saldırı = ZERO-DAY. Bu yüzden ifşa sırası önemlidir.

- **Zero-day:** satıcının bilmediği/yamasının olmadığı açık.
- **Sorumlu ifşa:** önce satıcıya bildir, ~90 gün bekle, sonra açıkla.
- **Yama boşluğu:** yama çıkınca açık kamuya açılır; güncellemeyen savunmasız (WannaCry).

Yaşam döngüsü: dört aşama

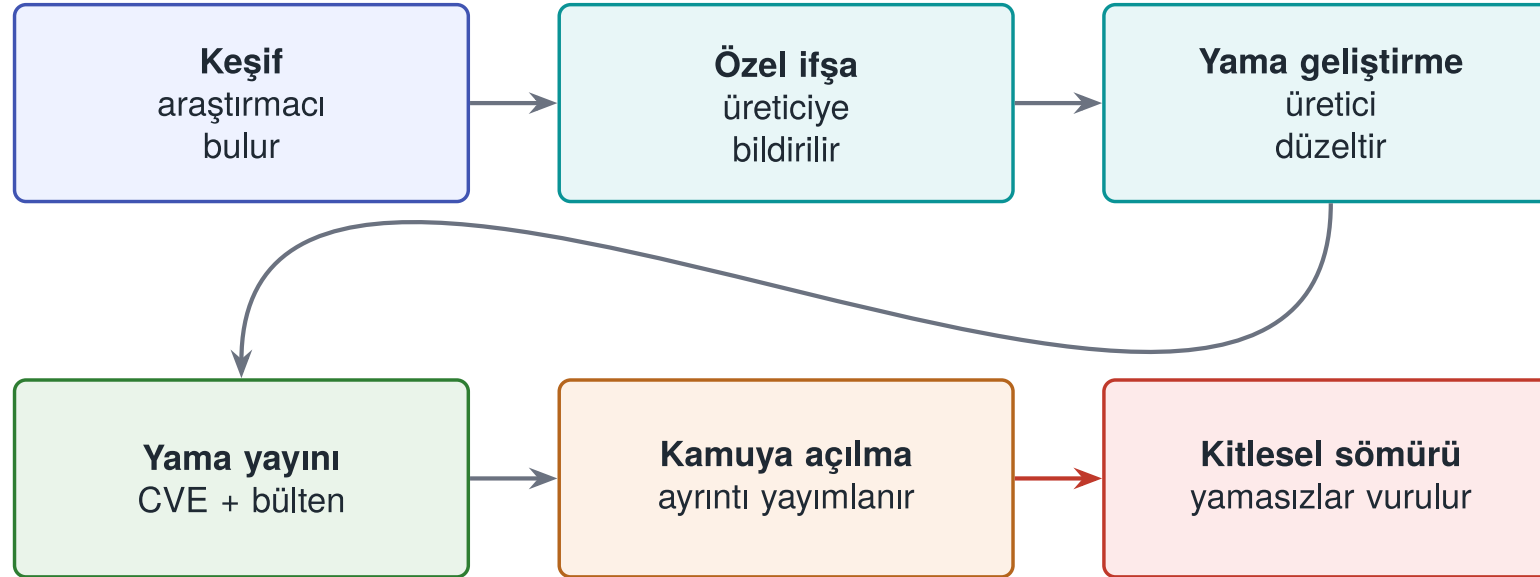
1. **Keşif:** açık bulunur (araştırmacı, saldırgan ya da satıcı).
2. **Bildirim:** araştırmacı satıcıya bildirir (ya da saldırgan sessizce kullanır).
3. **Yama:** satıcı düzeltmeyi yazıp yayımlar.
4. **Yayın:** yama kullanıcıya ulaşır, **uygulanırsa** risk biter.

Her ok arasında geçen süre, saldırgana verilen **zaman penceresidir**.

Açığın yaşam çizgisi — şema

Bir açığın yaşam çizgisinde kim ne yapar?

en tehlikeli aralık: yama çıktı ama kurulmadı



WannaCry'ın dersi tam burada: yama saldırıdan haftalar önce vardı.

Sorumlu ifşa vs tam ifşa

	Sorumlu ifşa (coordinated)	Tam ifşa (full)
Kim önce bilir?	Satıcı	Herkes aynı anda
Bekleme	~90 gün (yama için)	Yok
Amaç	Kullanıcıyı korurken satıcıyı harekete geçirmek	Satıcıyı zorlamak
Risk	Düşük (yama varken açıklanır)	Yüksek (yama yokken açık bilinir)

Sektörde tartışmalı olan **tam ifşadır**.

Hata ödül programları (bug bounty)

- Şirketlerin, açığı **sorumlu biçimde** bildiren araştırmacılara **ödül** verdiği programlar.
- Amaç: araştırmacıyı tam ifşa yerine **sorumlu ifşaya** yönlendirmek.
- Araştırmacı için kazanç; şirket için **erken uyarı**.
- Büyük teknoloji şirketlerinin çoğunda artık standart bir uygulamadır.

Sahada süreç: bir açık bildirilirse ne olur?

Bir ürün ekibi için yaşam döngüsü bir **süreçtir**:

- Bildirim **kanalı**: `security.txt`, güvenlik e-postası.
- Bir **sorumlu** kişi/ekip.
- Yama **SLA'sı**: ne kadar sürede düzeltilecek?
- Sürüm/yama **duyuru** mekanizması.

Bir değerlendirme yalnız kodu değil, bu **süreci** de sorar.

Tam ifşa neden tartışmalı?

Soru: Tam ifşa (full disclosure) neden riskli kabul edilir?

Cevap: Açık, **yama yokken** herkese duyurulur. Satıcıyı harekete geçirmeye zorlar, ama aynı zamanda saldırganlara da **hazır bir yol haritası** verir — kullanıcılar savunmasızken.

Kontrol listesi — zafiyet sınıflandırma

- [] CWE'nin soyutlama düzeylerini (pillar, class, base, variant) ve "en somut girdi" kuralını biliyorum.
- [] CWE, CVE ve CVSS'i birer cümleyle **ayırabiliyorum**.
- [] Bir bulguya CVSS v3.1 vektörü kurup puanı **elle** hesaplayabiliyorum.
- [] EPSS, KEV ve CVSS'in **farklı sorulara** cevap verdiğini açıklayabiliyorum.
- [] Zafiyet yaşam döngüsünü, sıfırinci gün, yama boşluğu ve sorumlu ifşayı anlatabiliyorum.

Dönem projesi: bu hafta (S4)

- [] S4 saldırı/tehdit tablonuzdaki **her satıra bir CWE** ekleyin.
- [] En kritik varlığa bir **saldırı ağacı** (Demo 04 biçimi); en ucuz yol + en etkili önlem.
- [] Kullandığınız bileşenlerin **CVE**'lerini bulup **CVSS** puanlayın.
- [] Uygun **erişim modelini** (DAC / MAC / RBAC) gerekçelendirin.

ID	Tehdit	Varlık	Yol	CWE	Önlem	Bölüm
----	--------	--------	-----	-----	-------	-------

Sınıf etkinlikleri

1. **Zararlıyı sınıfla, katmanı seç:** altı senaryoda tür ve ilk yakalayan katman
2. **Saldırı ağacı düellosu:** karşı grup, ağaçta olmayan daha ucuz bir yol arar
3. **Matristen modele:** aynı politikayı ACL, yetenek listesi, BLP/Biba etiketiyle yaz
4. **Bulgudan yama sırasına:** CWE + CVSS + EPSS + KEV + SSVC ile beş bulguyu sırala
5. **Denetim kaydını kurcala:** Demo 11 zincirinde değiştirme/silme/sondan kesmeyi dene
6. **Kod okuma turnuvası:** dört koda CWE, saldırı ve düzeltme yaz
7. **Olay otopsisı:** Heartbleed/WannaCry/Log4Shell için CVE-CWE-CVSS-tarih doldur

Kendini sinama (seçme)

1. Virüs, solucan, truva atı farkı?
2. Polimorfizm $\neq$ metamorfizm: fark?
3. Yüksek entropi zararlıyı **kanıtlar** mı?
4. BLP iki kuralı ve neyi korur?
5. Biba, BLP'den nasıl ayrılır?
6. DAC ile MAC farkı?
7. CWE ile CVE farkı?
8. CVSS Temel puan neyi ölçmez?

Kendini sınaama — cevaplar (1–4)

1. **Virüs** başka programa bulaşır; **solucan** kendi başına ağdan kopyalar; **truva atı** faydalı görünüp gizli zararlı taşır.
2. **Polimorfik** her kopyada kendini farklı **şifreler**; **metamorfik** kodunu **yeniden yazar**.
3. **Hayır**. Yüksek entropi bir **ipucudur** (şifreli/paketli); tek başına zararlıyı kanıtlamaz (meşru sıkıştırma da yüksek entropili).
4. **BLP**: yukarı okuma yok + aşağı yazma yok → **gizliliği** korur.

Kendini sınaama — cevaplar (5–8)

5. **Biba** BLP'nin tersidir: aşağı okuma yok + yukarı yazma yok → **bütünlüğü** korur.
6. **DAC** izinleri sahibi belirler (esnek, sızıntıya açık); **MAC** sistemin zorunlu etiketli kuralları (katı).
7. **CWE** zayıflık **türü** (ör. CWE-416); **CVE** belirli bir ürünlerdeki **somut** açık.
8. **CVSS Temel puan** olabilirliği/etkiyi ölçer ama **istismarın sahada gerçekleşme olasılığını** (tehdit/ortam) tek başına ölçmez.

Kendini sinama (devam)

- 9. İmza ve sezgisel tespitin güçlü/zayıf yanı?
- 10. Günlük enjeksiyonu (CWE-117) nasıl önlenir?
- 11. Sorumlu ifşa nedir?

Kendini sinama — cevaplar (9–11)

9. **İmza** hızlı ama yeniyi kaçıır; **sezgisel** yeniyi bulur ama yanlış alarm üretir.
10. Günlüğe yazılan kullanıcı verisindeki satır sonu/kontrol karakterlerini **temizle/kaçır**; yapılandırılmış günlük kullan.
11. Açığı önce **üreticiye bildirip** yama için süre tanımak, sonra **kamuya açıklamak**.

Sözlük

Terim	Anlam
Polimorfik	Her kopyada farklı şifreli
İmza/sezgisel	Bilinen iz / şüpheli davranış
Entropi	Rastgelelik ölçüsü
DAC/MAC/RBAC	Erişim denetim modelleri
BLP/Biba	Gizlilik / bütünlük modeli
CWE/CVE/CVSS	Tür / somut açık / puan

Sözlük (devam)

Terim	Anlam
EPSS	Açığın 30 gün içinde istismar edilme olasılığı
KEV	Kanıtlanmış istismar edilen açıklar kataloğu
SSVC	İstismar/etki/yaygınlık ile Track-Act kararı
Sorumlu / tam ifşa	Önce satıcıya bildirme / hemen kamuya açma
Kimlik doğrulama / yetkilendirme	"Sen kimsin?" / "Bunu yapabilir misin?"
Güvenli varsayılan	Politika belirsizse cevap ret (fail-closed)

Bir sonraki hafta

3. hafta — Veri Güvenliği: Aktarımda, Beklemede, Kullanımda

Bu hafta zararlı yazılımı tespit ederken kullandığımız **entropi** ölçütünü, gelecek hafta kriptografik rastgele sayı üreticinin (CSPRNG) girdisindeki düzensizliği değerlendirmek için yeniden kullanacağız. 3. haftada şifreleme temellerini, TLS 1.3 ve sertifika sabitlemeyi, AES-GCM ile yerel şifrelemeyi ve güvenli silmeyi işleyeceğiz (Kaynak: Viega & Messier, Tarif 9.1–9.3, 10.7–10.9, 4.9–4.11, 13.2).

Bu haftanın özeti: tehdidi **sınıflandır**, erişimi **modelle**, açığı **kataloglarla** (CWE/CVE) izle ve ciddiyetini (**CVSS**) ölç. Bu çerçeve, sonraki haftalardaki teknik korumaların zeminidir.

Bu hafta CWE ve CVSS ile önceliklendirdiğimiz zafiyet dili, **12. haftada** sızma testi planlamasının girdisi olarak yeniden karşımıza çıkacak.